

MANIPAL INSTITUTE OF TECHNOLOGY
Bengaluru – 560064

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING



CERTIFICATE

This is to certify that Ms./Mr. Reg.

No. Section: Roll No: has satisfactorily

completed the lab exercises prescribed for Digital Systems Lab [CSE_2162] of Second Year B. Tech. Degree

in Computer Science and Engineering at MIT Bengaluru, in the academic year 2022- 2023.

Date:

Signature

Faculty in Charge

CONTENTS

LAB NO.	TITLE	PAGE NO.
	Course Objectives and Outcomes	i
	Evaluation plan	i
	Instructions to the Students	iii
	Sample Lab Observation note preparation	iv
1	Introduction to Verilog	1-21
2	Realize all logic gates	22-23
3	Simplification using k-maps	24-25
4	Multilevel Synthesis	26-27
5	Arithmetic Circuits	28-31
6	Multiplexers	32-34
7	Decoders and encoders	35-37
8	Comparators and Code Converters	38-41
9	Flip Flops and Registers	42-45
10	Counters	46-49
11	Realize 4-bit Binary to Grey Converters	50-51
12	Simple Processor Design	52-58
	References	59
	Viva Questions	60-62

Course Objectives

- To develop the skills of implementing logic circuits using Verilog.
- Simplify the logical expressions and implement using logic gates.
- Design and analyze the combinational and sequential circuits, simple systems.
- Relate theoretical concepts to practical applications like a multiplexer, encoder, decoder, code converter, counter, shift register applications.

Course Outcomes

At the end of this course, students will be able to

- Simplify logical expressions and simulate using Verilog.
- Design and analyze arithmetic circuits and combinational circuits using multiplexers, encoders, and decoders.
- Construct multiplexers, decoders, encoders and simulate them to suit various practical applications.
- Design and simulate sequential circuits using Verilog.
- Design and simulate simple processors.

Evaluation Plan

- Internal Assessment Marks: 60%
 - ✓ Continuous evaluation component (for each evaluation):10 marks
 - ✓ The assessment will depend on punctuality, program execution, maintaining the observation note and answering the questions in viva voce.
 - ✓ Total marks of the 12 evaluations reduced to marks out of 60.
- End semester assessment of 2 hour duration: 40 %

INSTRUCTIONS TO THE STUDENTS

Pre- Lab Session Instructions

1. Students should have a separate observation book and the required stationery for every lab session.
2. Be on time and follow the institution dress code.
3. Must sign in the log register provided.
4. Make sure to occupy the allotted seat and answer the attendance.
5. Adhere to the rules and maintain the decorum.

In- Lab Session Instructions

- Follow the instructions on the allotted exercises.
- Show the program and results to the instructors on completion of experiments.
- On receiving approval from the instructor, copy the program and results in the Labrecord.
- Prescribed textbooks and class notes can be kept ready for reference if required.

General Instructions for the exercises in Lab

- Implement the given exercise individually and not in a group.
- The observation book should be complete with proper design, logical diagrams, truth tables and waveforms related to the experiment they perform.
- Plagiarism (copying from others) is strictly prohibited and would invite severe penalties in evaluation.
- The exercises for each week are divided into three sets:
 - Solved exercise.
 - Lab exercises - to be completed during lab hours.
 - Additional Exercises - to be completed outside the lab or in the lab to enhance the skill.
- Questions for lab tests and examinations are not necessarily limited to the questions in the manual but may involve some variations and/or combinations of the questions.
- A sample note preparation is given as a model for observation.

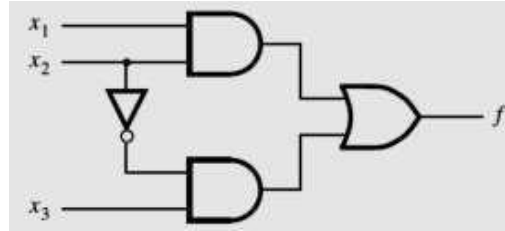
THE STUDENTS SHOULD NOT

- Bring mobile phones or any other electronic gadgets to the lab.
- Go out of the lab without permission.

SAMPLE LAB OBSERVATION NOTE PREPARATION

Title: Introduction to Verilog

1. Write Verilog code to implement the following circuit using the continuous assignment.



Aim: To write Verilog code, Truth table, and waveform for the above circuit.

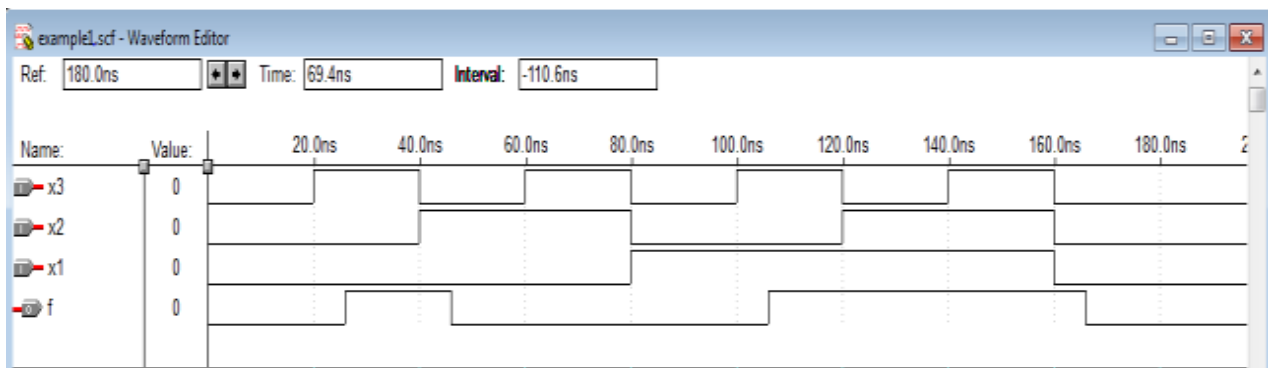
Verilog code:

```
module example2(x1,x2,x3,f);  
    input x1,x2,x3;  
    output f;  
    assign f=(x1 & x2)| (~x2 & x3);  
endmodule
```

Truth table:

x1	x2	x3	F
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	1

Waveform:



INTRODUCTION TO VERILOG

Objectives:

In this lab, student will be able to

1. Learn the basic concepts of logic circuits and analyze the logic network.
2. Write the Truth table and Timing diagram.
3. Understand different representation of logic circuits in Verilog.
4. Learn the different tools available in the CAD system.
5. Write and simulate logic circuits using Verilog.

I. Basic concepts of Logic Circuits

Logic Circuits

- Perform operations on digital signals.
- Signal values are restricted to a few discrete values.
- In binary logic circuits, there are only two values, 0 and 1.

Logic Gates and Networks

- Each logic operation can be implemented electronically with transistors, resulting in a circuit element called a logic gate.
- It has one or more inputs and one output that is a function of its inputs.
- It is often convenient to describe a logic circuit by drawing a circuit diagram, or schematic, consisting of graphical symbols representing the logic gates.

The graphical symbols for the AND, NOT and OR gates are shown in Fig. 1.1, 1.2 and 1.3 respectively.

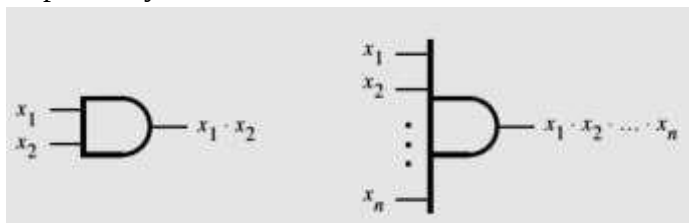


Figure 1.1

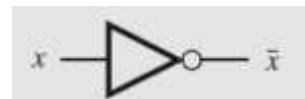


Figure 1.2

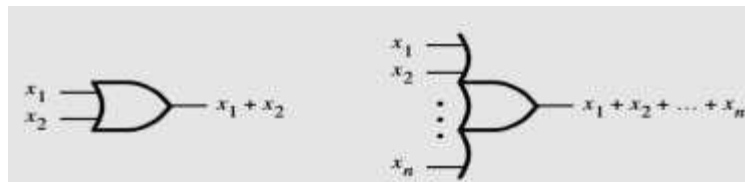


Figure 1.3

- A larger circuit is implemented by a network of gates, as shown in Fig. 1.4

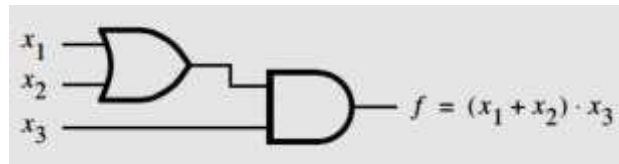


Figure 1.4

x_1	x_2	$x_1 \cdot x_2$	$x_1 + x_2$
0	0	0	0
0	1	0	1
1	0	0	1
1	1	1	1
		AND	OR

Figure 1.5 Truth Table

- The operations AND, OR etc can also be defined in the form of a table as shown in Figure 1.5.
- The first two columns (to the left of the heavy vertical line) give all four possible combinations of logic values that the variables x_1 and x_2 can have.
- The next column defines the AND operation for each combination of values of x_1 and x_2 , and the last column defines the OR operation.
- In general, for n input variables the truth table has 2^n rows.

II. Analysis of a Logic Network

- Determining the function performed by an existing logic network is referred to as the Analysis process.
- The reverse task of designing a new network that implements a desired functional behavior is referred to as the Synthesis process.
- To determine the functional behavior of the network in Fig. 1.6, we can consider what happens if we apply all possible values to input signals x_1 and x_2 . The analysis of these input values at various intermediate points is shown in Fig. 1.7.

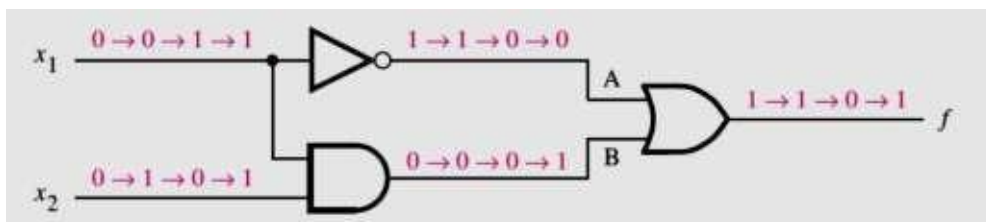


Figure 1.6. functional behavior of the network

x1	x2	A	B	f
0	0	1	0	1
0	1	1	0	1
1	0	0	0	0
1	1	0	1	1

Fig. 1.7. Input values at various intermediate points

Timing Diagram

- The information in Fig. 1.7 can be presented in graphical form, known as a timing diagram, as shown in Fig. 1.8.
- The figure shows the waveforms for the inputs and output of the network, as well as for the internal signals at the points labeled A and B.

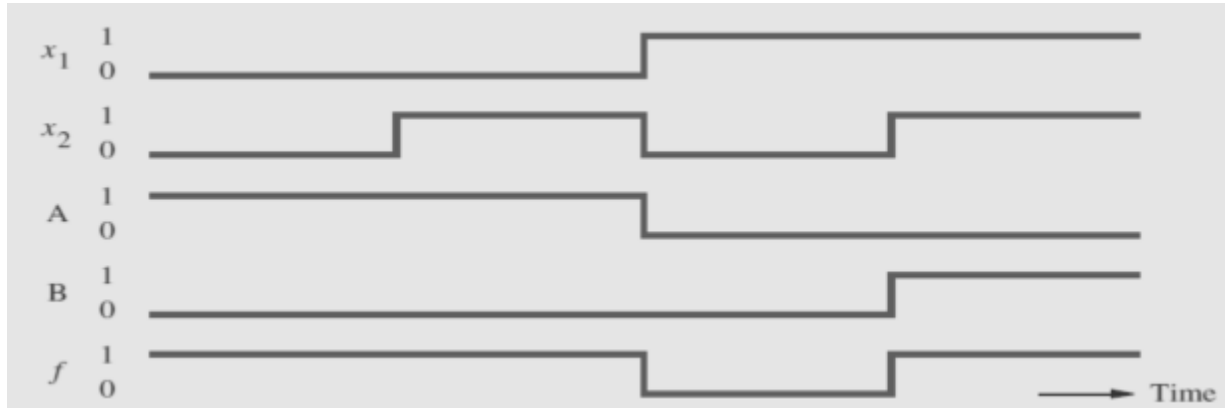


Figure 1.8. Timing diagram of the above circuit

Functionally Equivalent Networks

- Going through the same analysis procedure, we find that the output 'g' in Fig. 1.9, changes in exactly the same way as f does in Fig. 1.6.
- Therefore, $g(x_1, x_2) = f(x_1, x_2)$, which indicates that the two networks are functionally equivalent.

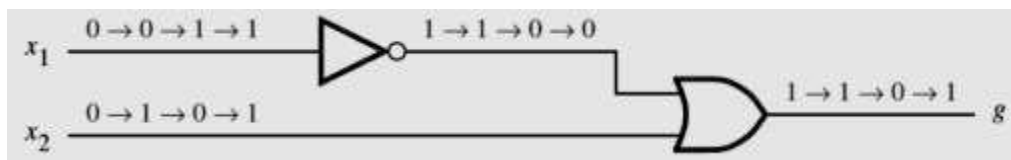


Figure 1.9

III. Introduction to CAD Tools

- Logic circuits are designed using CAD tools that automatically implement synthesis techniques.
- CAD system includes tools for design entry, synthesis and optimization, simulation and physical design.

Design Entry

- The starting point in the process of designing a logic circuit is the conception of what the circuit is supposed to do and the formulation of its general structure.
- The first stage of this process involves entering into the CAD system a description of the circuit being designed. This stage is called design entry.
- For design entry, we are writing source code in a hardware description language.

Hardware Description Languages

- A hardware description language (HDL) is similar to a typical computer programming language except that an HDL is used to describe hardware rather than a program to be executed on a computer.
- Two HDLs are IEEE standards: Verilog HDL and VHDL.

Why uses Verilog

- Supported by most companies that offer digital hardware technology.
- Verilog provides design portability. A circuit specified in Verilog can be implemented in different types of chips and with CAD tools provided by different companies, without changing the Verilog specification.
- Both small and large logic circuit designs can be efficiently represented in Verilog code.

Functional Simulation

- The functional simulator tool verifies that the designed circuit functions as expected.
- It uses two types of information.
 - First, the user's initial design is represented by the logic equations generated during synthesis.
 - Second, the user specifies the valuations of the circuit's inputs that should be applied to these equations during the simulation.
- For each valuation, the simulator evaluates the outputs produced by the expressions.
- The results of simulations are usually provided in the form of a timing diagram that the user can examine to verify that the circuit operates as required.

Timing Simulation

- When the values of inputs to the circuit change it takes a certain amount of time before a corresponding change occurs at the output. This is called a propagation delay of the circuit.

IV. Representation of Digital Circuits in Verilog

- **Structural representation-** A larger circuit is defined by writing code that connects simple circuit elements together.
- **Behavioral representation-** Describing a circuit by using logical expressions and programming constructs that define the behavior of the circuit but not its actual structure in terms of gates.

Structural Specification of Logic Circuits

- A gate is represented by indicating its functional name, output, and inputs. Different logic gates are shown in Table 1.1

For example,

- A two-input AND gate, with inputs x1 and x2 and output y, is denoted as **and** (y, x1, x2);
- A four-input OR gate is specified as **or** (y, x1, x2, x3, x4);
- The NOT gate is given by **not** (y, x); implements $y = x'$.

Table 1.1. Various Logic gates

Name	Description	Usage
and	$f = (a \cdot b \cdot \dots)$	and (f, a, b, ...)
nand	$f = \overline{(a \cdot b \cdot \dots)}$	nand (f, a, b, ...)
or	$f = (a + b + \dots)$	or (f, a, b, ...)
nor	$f = \overline{(a + b + \dots)}$	nor (f, a, b, ...)
xor	$f = (a \oplus b \oplus \dots)$	xor (f, a, b, ...)
xnor	$f = (a \odot b \odot \dots)$	xnor (f, a, b, ...)
not	$f = \overline{a}$	not (f, a)

Verilog Module

- It is a circuit or subcircuit described with Verilog code.
- The module has a name, **module_name**, which can be any valid identifier, followed by a list of ports.
- The term port refers to an input or output connection in an electrical circuit. The ports can be of type **input**, **output**, or **inout** (bidirectional), and can be either scalar or vector.

The General Form of a Module

```
module module name [(port name{, port name})];  
    [parameter declarations]  
    [input declarations]  
    [output declarations]  
    [inout declarations]  
    [wire or tri declarations]  
    [reg or integer declarations]  
    [function or task declarations]  
    [assign continuous assignments]  
    [initial block]  
    [always blocks]  
    [gate instantiations]  
    [module instantiations]  
endmodule
```

Documentation in Verilog Code

- Documentation can be included in Verilog code by writing a comment. A short comment begins with the double slash, //, and continues to the end of the line. A long comment can span multiple lines and is contained inside the delimiters /* and */.

White Space

- White space characters, such as SPACE and TAB, and blank lines are ignored by the Verilog compiler.
- Multiple statements can be written on a single line.
- Placing each statement on a separate line and using indentation within blocks of code, such as an **if-else** statement are good ways to increase the readability of code.

Signals in Verilog Code

- A signal in a circuit is represented as a net or a variable with a specific type.
- A net or variable declaration has the form
 type [range] signal_name{, signal_name};
- The signal_name is an identifier
- The range is used to specify vectors that correspond to multi-bit signals

Signal Values and Numbers

- Verilog supports scalar nets and variables that represent individual signals and vectors that correspond to multiple signals.
- Each individual signal can have four possible values:
0 = logic value 0 1 = logic value 1
z = tri-state (high impedance) x = unknown value
- The value of a vector variable is specified by giving a constant of the form [size]['radix]constant where size is the number of bits in the constant, and radix is the number base. Supported radices are
d = decimal b = binary h = hexadecimal o = octal
- Some examples of constants include
0 the number 0 10 the decimal number 10
'b10 the binary number 10 = (2)₁₀ 'h10 the hex number 10 = (16)₁₀
4'b100 the binary number 0100 = (4)₁₀

Nets

Verilog defines a number of types of nets.

- A net represents a node in a circuit.
- For synthesis purposes, the only important nets are of **wire** type.
- For specifying signals that are neither inputs nor outputs of a module, which are used only for internal connections within the module, Verilog provides the **wire** type.

Identifier Names

- Identifiers are the names of variables and other elements in Verilog code.
- The rules for specifying identifiers are simple: any letter or digit may be used, as well as the _ underscore and \$ characters.
- An identifier must not begin with a digit and it should not be a Verilog keyword.
 - Examples of legal identifiers are f, x1, x, y, and Byte.
 - Some examples of illegal names are 1x, +y, x*y, and 258
- Verilog is case sensitive; hence k is not the same as K, and BYTE is not the same as Byte.

Verilog Operators

- Verilog operators are useful for synthesizing logic circuits.
- Table 1.2 lists these operators in groups that reflect the type of operation performed.

Table 1.2. Operators and their operations

Operator type	Operator Symbols	Operation Performed	Number of operands
Bitwise	~	1's complement	1
	&	Bitwise AND	2
		Bitwise OR	2
	^	Bitwise XOR	2
	~^ or ^~	Bitwise XNOR	2
Logical	!	NOT	1
	&&	AND	2
		OR	2
Reduction	&	Reduction AND	1
	~&	Reduction NAND	1
		Reduction OR	1
	~	Reduction NOR	1
	^	Reduction XOR	1
	~^ or ^~	Reduction XNOR	1
Arithmetic	+	Addition	2
	-	Subtraction	2
	-	2's complement	1
	*	Multiplication	2
	/	Division	2
Relational	>	Greater than	2
	<	Lesser than	2
	>=	Greater than or equal to	2
	<=	Lesser than or equal to	2
Equality	==	Logical equality	2
	!=	Logical inequality	2
Shift	>>	Right shift	2
	<<	Left shift	2
Concatenation	{,}	Concatenation	Any number
Replication	{{}}	Replication	Any number
Conditional	?:	Conditional	3

Introduction to VIVADO tool

Aim:

To study the simulation of Verilog code using Vivado tool.

Introduction:

Xilinx Tool is a suite of software tools used for the implementation of digital circuits using Xilinx Field Programmable Gate Array (FPGA) or Complex Programmable Logic Device (CPLD). The design procedure consists of (a) design entry, (b) synthesis and implementation of the design, (c) functional simulation and (d) testing and verification.

Digital design can be entered in various ways - using *schematic entry* tool, using a Hardware Description Language (HDL) - Verilog or VHDL or a combination of both. Here, we will consider the Verilog HDL based design flow.

A Verilog input file typically consists of the following segments:

- **Header:** module name, list of input and output ports.
- **Declarations:** input and output ports, registers and wires.
- **Logic Descriptions:** equations, state machines and logic functions.
- **End:** endmodule

All the designs must be specified using the above given Verilog input format.

Getting Started

To start VIVADO, double-click the desktop icon,,



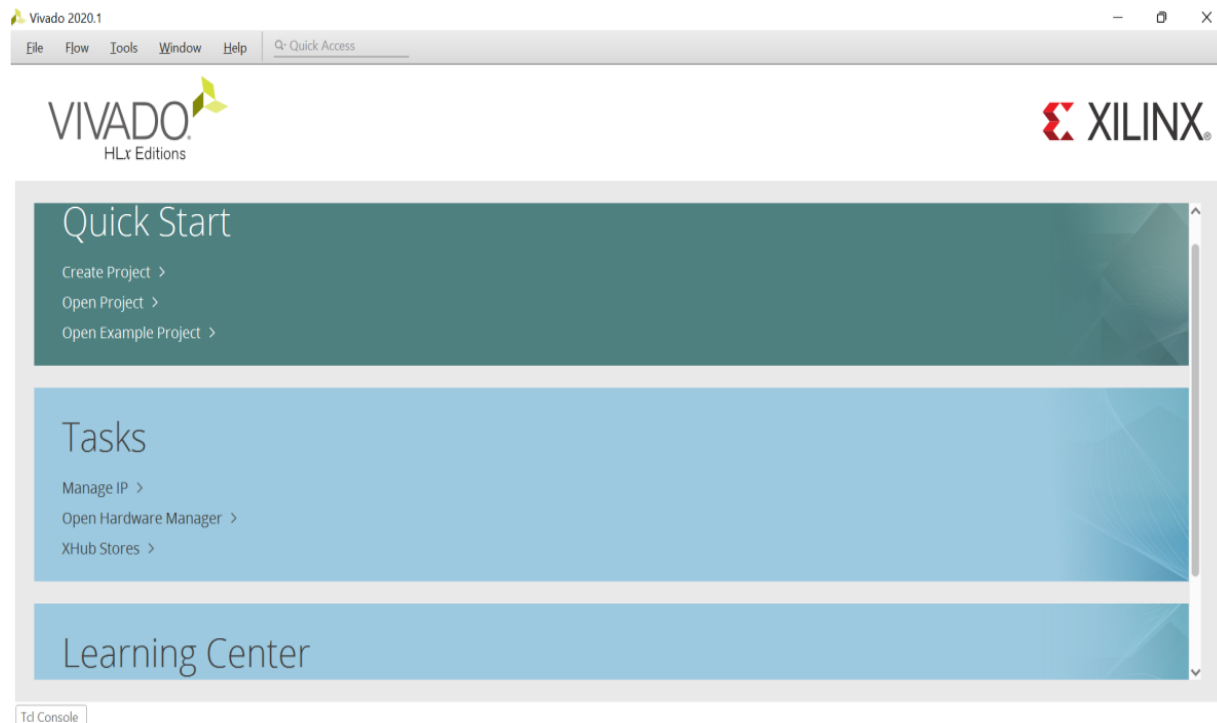
or start VIVADO from the Start menu by selecting:

Start All Programs VIVADO 2020—• Project Navigator Note:

Your start-up path is set during the installation process and may differ from the one above.

Create a New Project

Double click on the Vivado icon on your desktop to open up the welcome window of the development tool (as shown below). Three main sections can be observed in this window: “Quick Start”, “Tasks”, and “Learning Center”.



To create a new project:

1. In the opening menu, as soon as the VIVADO opens it notifies the Quick start option. **Quick Start>Create Project...** The New “**Create a New VIVADO Project**” Wizard appears.
2. Type **Next**, immediately, you see a below screen. Name your project, 'tst_1' and it will be in the folder.

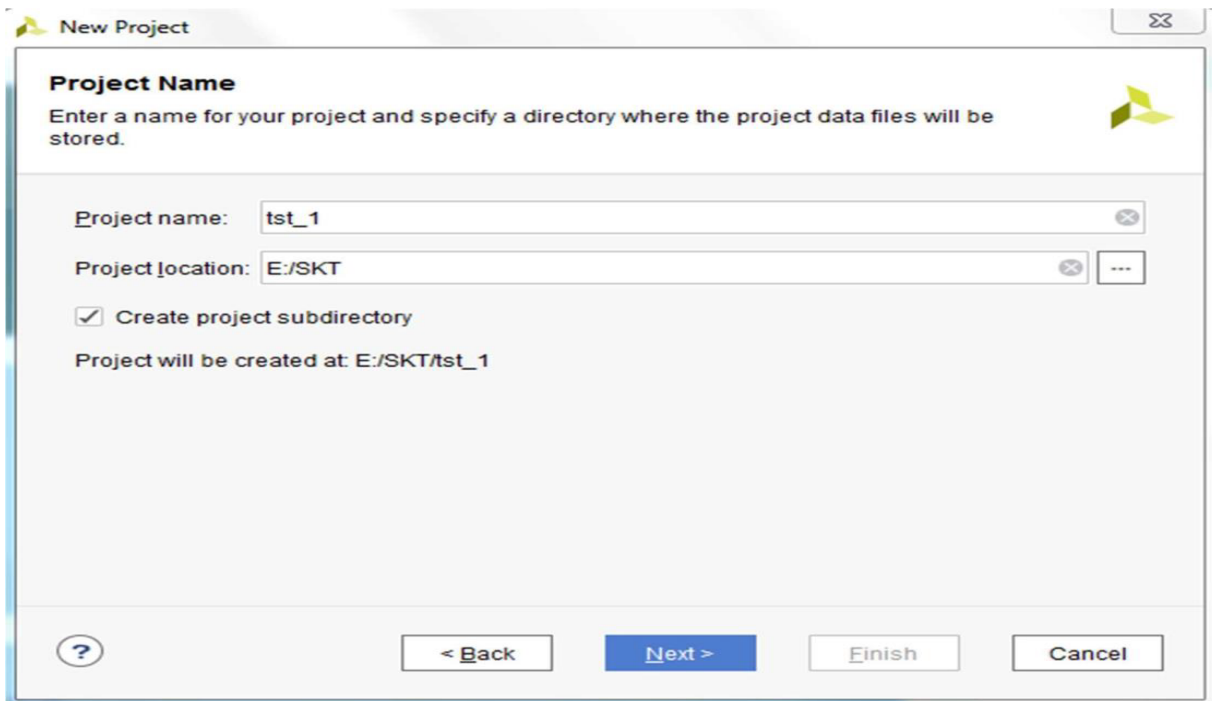


Fig.1.1: The New Project Wizard

3. In the Project Name field enter the desired Project Name. Enter or browse to a project location (directory path) for the new project, check the box project subdirectory, and press next.
4. A new window will open which shows the **project type**. **Click on RTL project**. In the next window, choose “RTL Project” as the project type and click next.

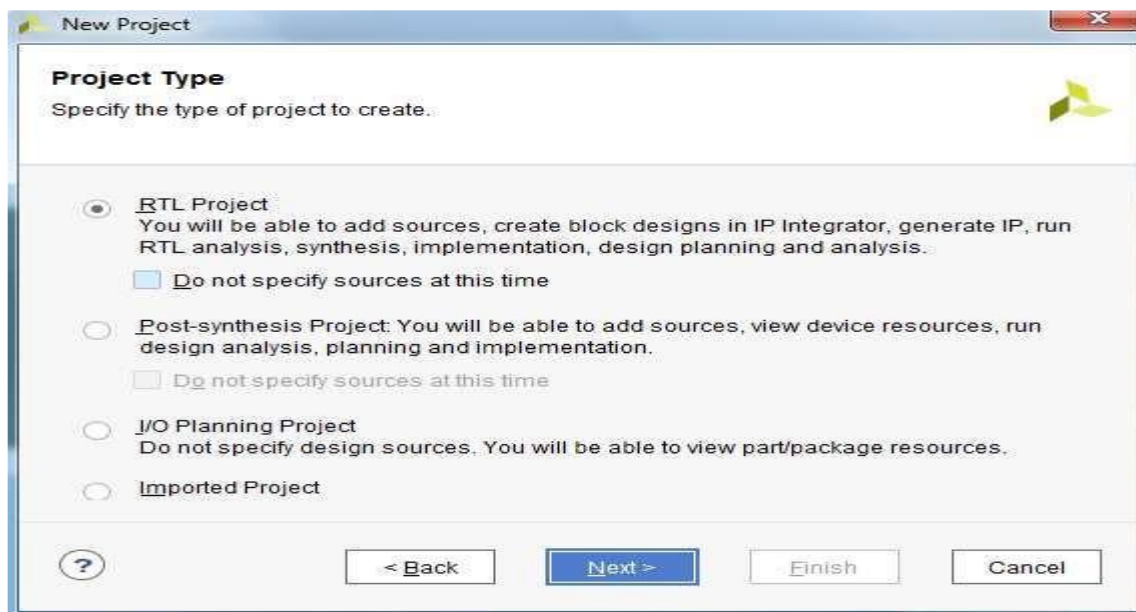


Fig.1.2: Project Type

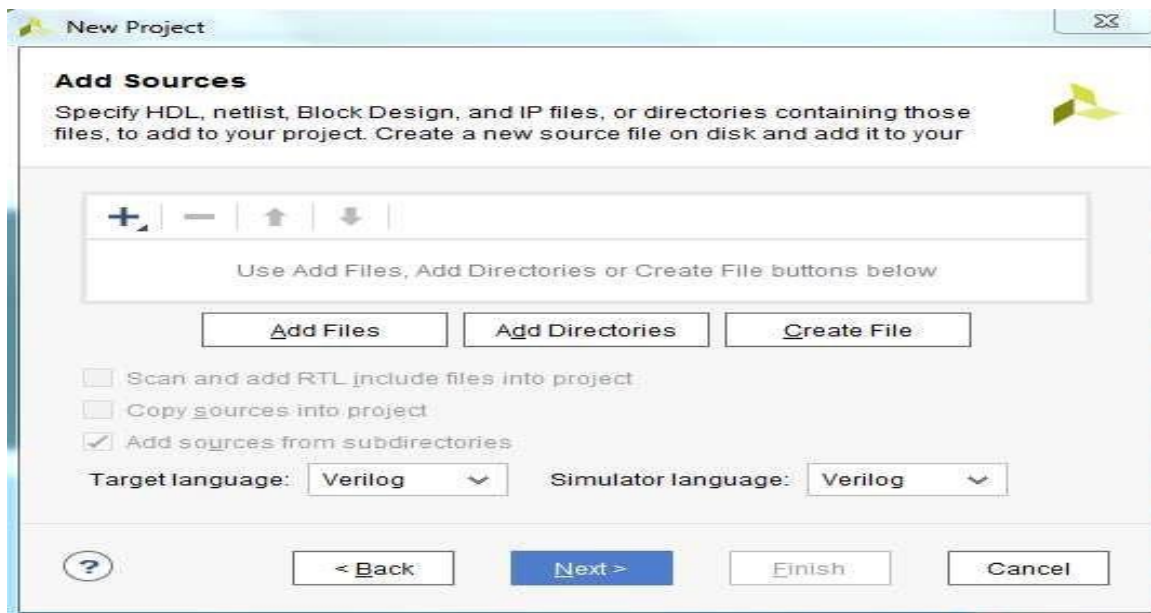


Fig.1.3: Add Sources

5. In the Add sources window click on create source to create a new Verilog file. If the Verilog file already exists, then click on add file and browse folder file.

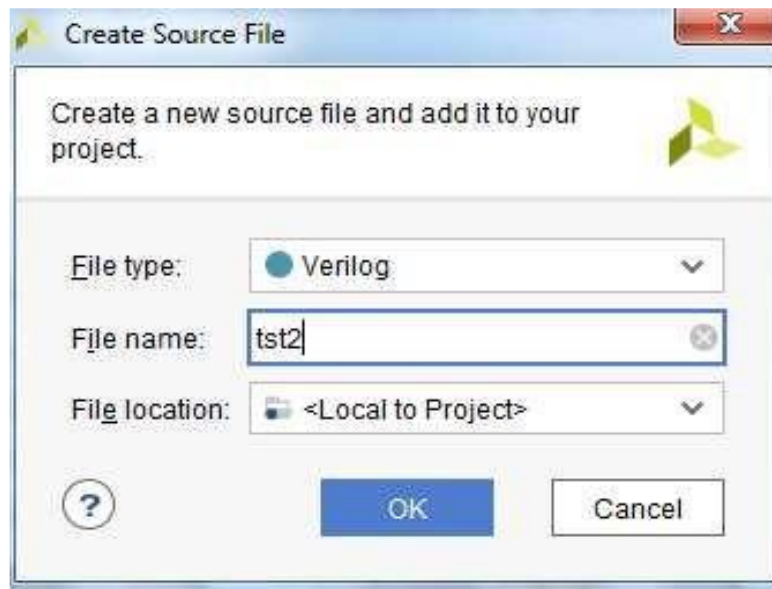


Fig.1.4(a): Create Source File

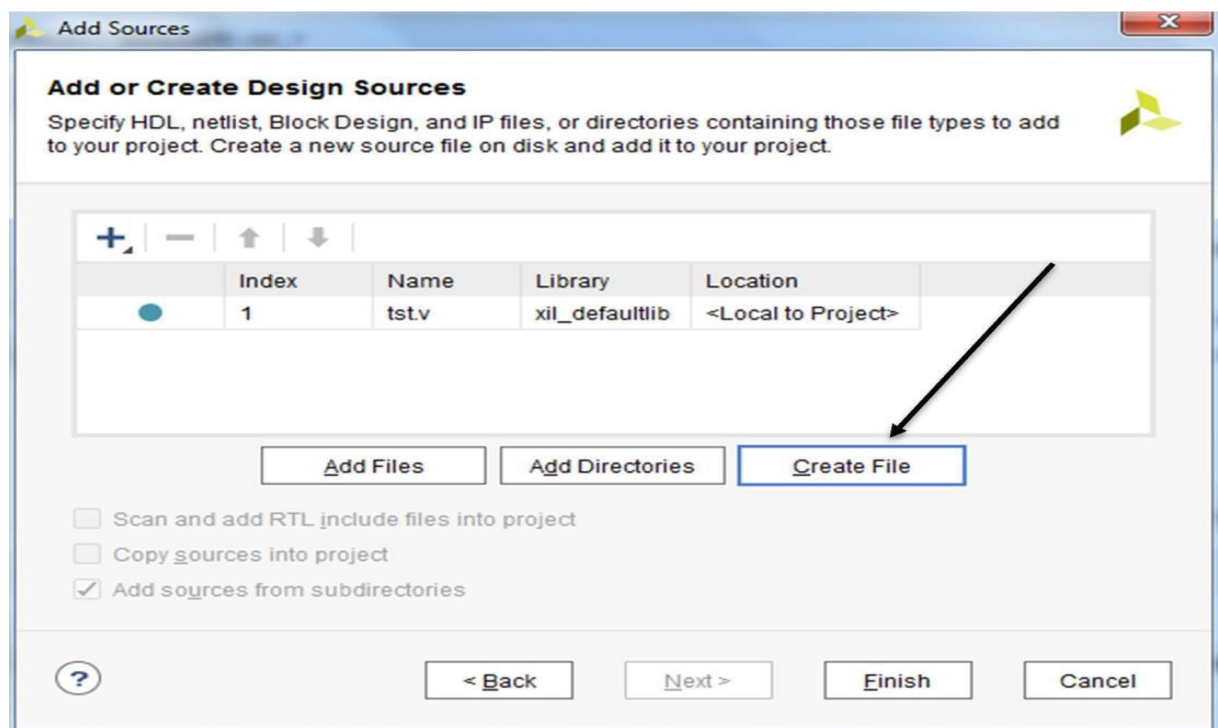


Fig.1.4(b): Create Source File

6. In the create source file window specify the hardware language as Verilog. Specify the **New file name** and click ok.

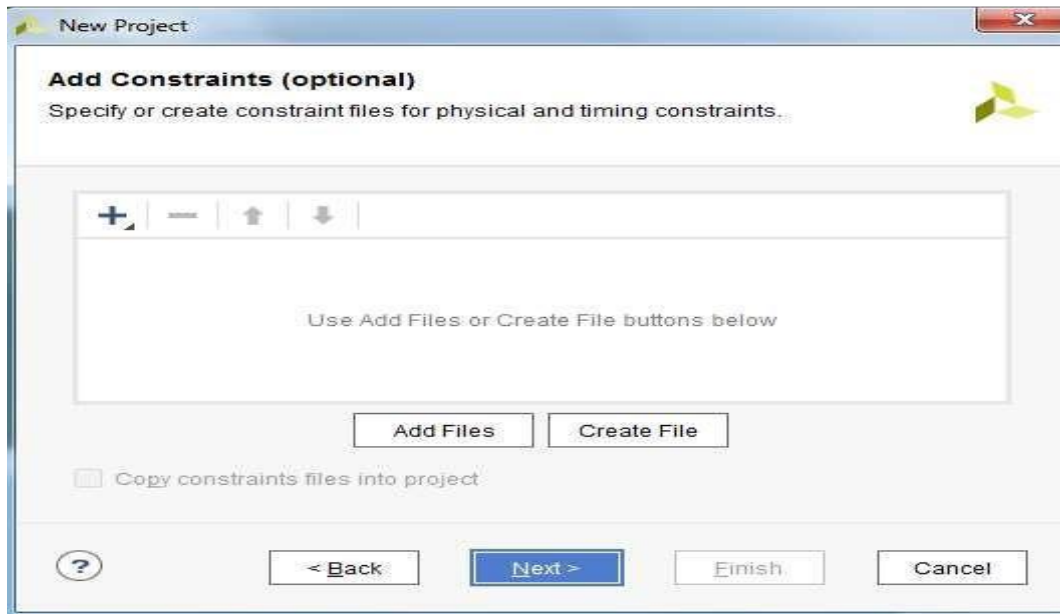


Fig.1.5: Add Constraints

This is an optional window click on next.

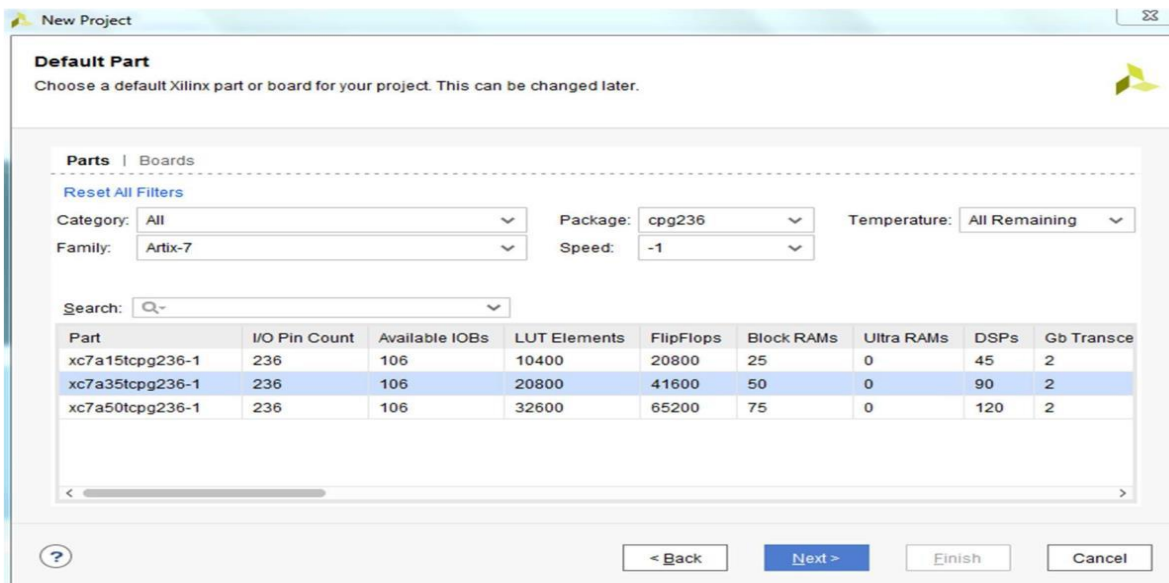


Fig.1.6: Default part

7. Fill in the properties as shown in Fig. 6: to select the board for the project.

◆ Product Category: **All**

◆ Family: **Artix -7**

◆ Speed: **-1**

◆ Package: **cpg236**

Click Next

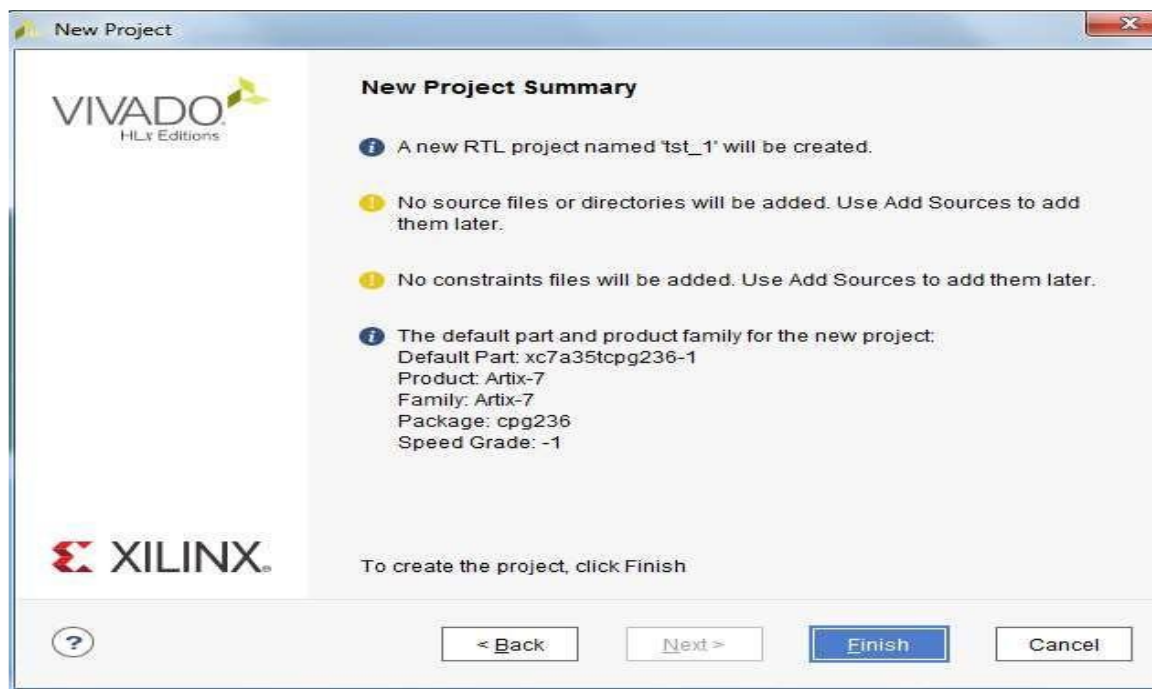


Fig.1.7: New Project Window

Click Finish to create the New Project.

8. New Project window will appear. Under Design sources of the Source window your file will appear in .v format. Then double click on the file to start writing your code.

If your file does not appear in the sources window, or if the Sources were not added previously then follow the below figure to add sources. After this step a new window will open. Click on Add Sources.

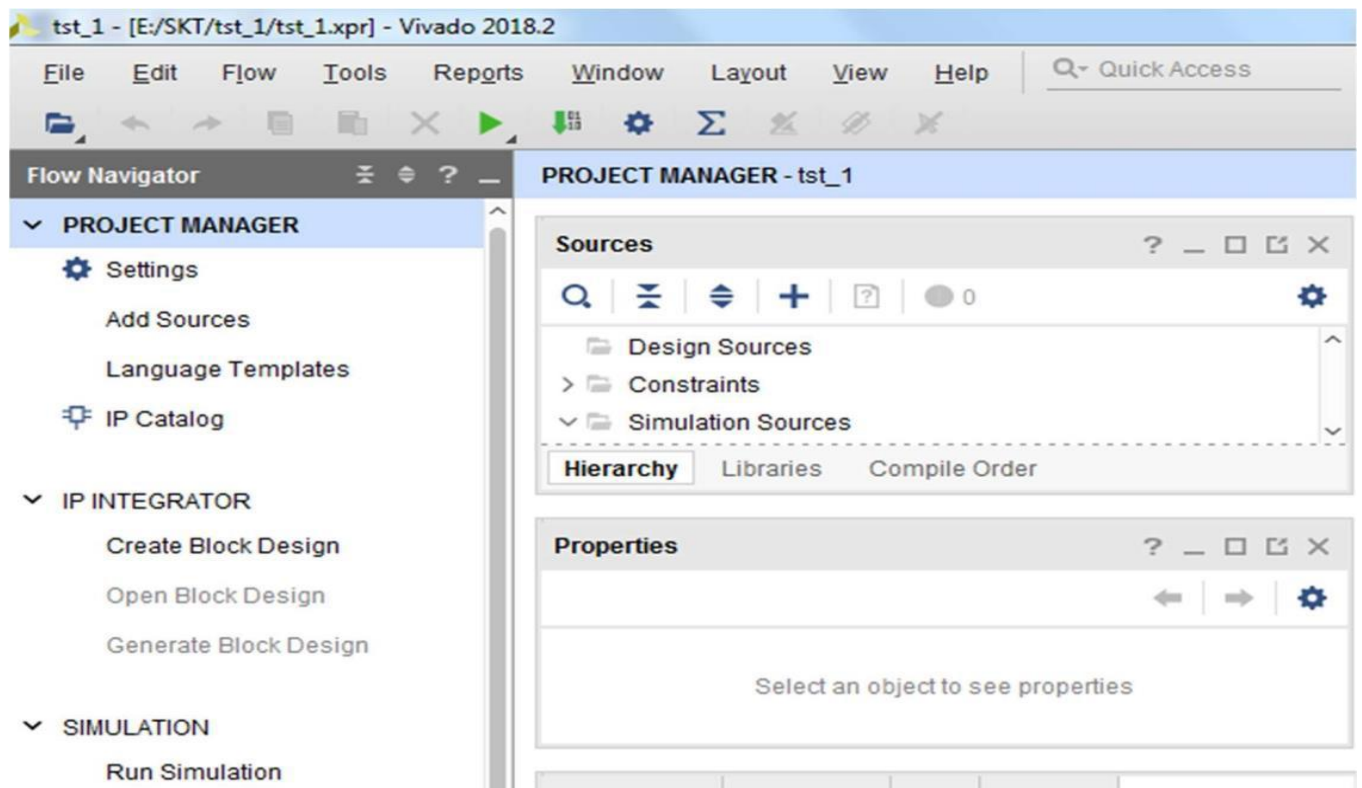


Fig.1.8: Selecting the source file.

9. Adding Sources: Click on add or create design sources and then click next.

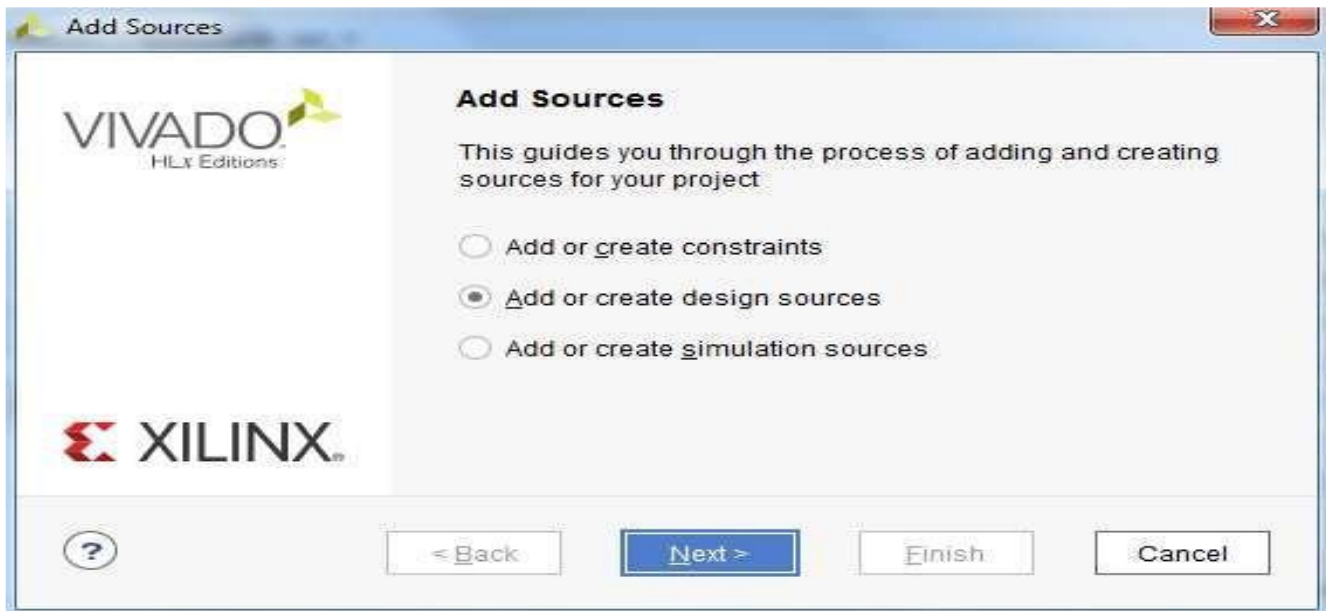
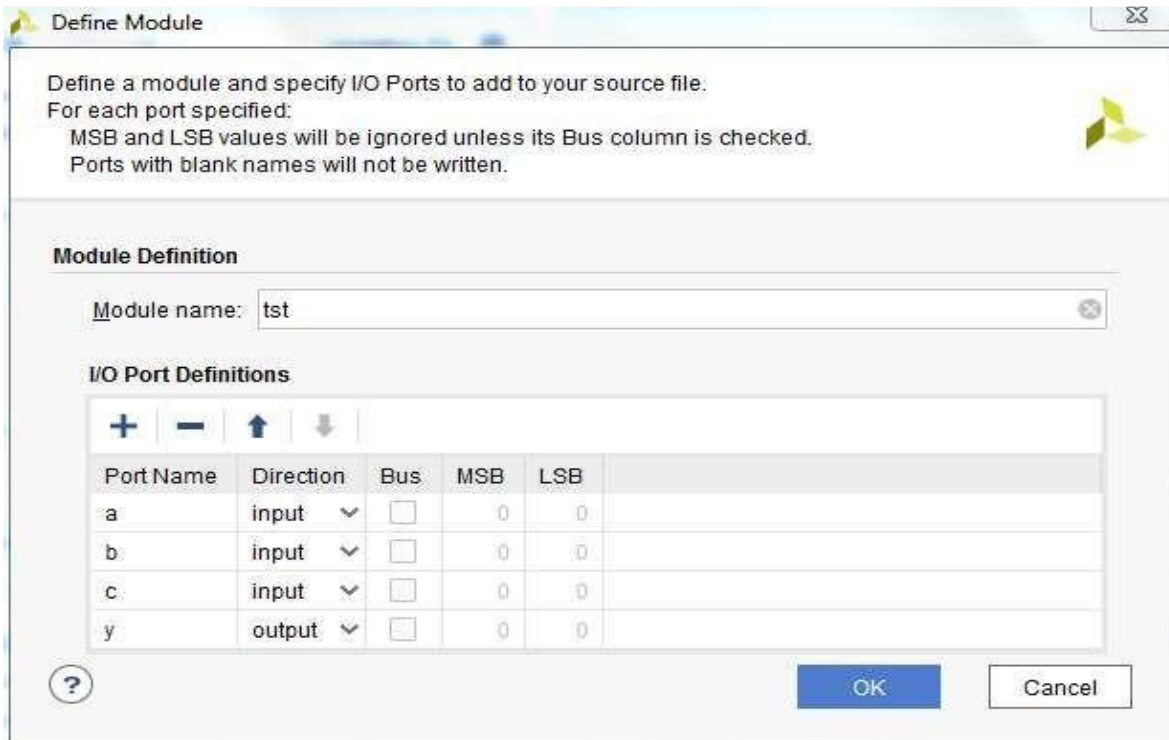


Fig.1.9: Add Sources

10. Add or Create Design source window will open and click on create file.
11. Click Finish.
12. The **Define module** requests input and outputs for the module. Specify your input and output variables and **Click OK**.



Define Module

Define a module and specify I/O Ports to add to your source file.
For each port specified:
MSB and LSB values will be ignored unless its Bus column is checked.
Ports with blank names will not be written.

Module Definition

Module name:

I/O Port Definitions

Port Name	Direction	Bus	MSB	LSB
a	input	☐	0	0
b	input	☐	0	0
c	input	☐	0	0
y	output	☐	0	0

Buttons: ? OK Cancel

Fig.1.10: I/O port definitions.

13. Double click the created tst2 to get an editor window, in which we write our program.

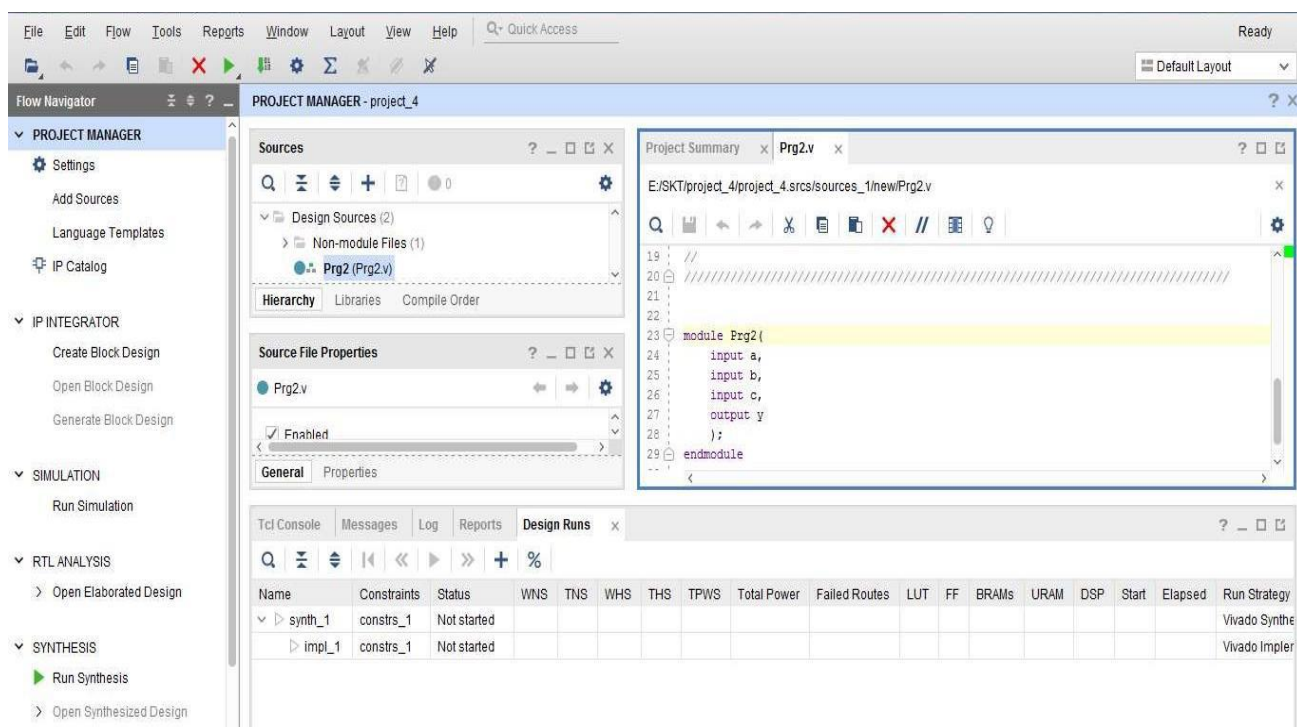


Fig.1.11(a): Writing the source code

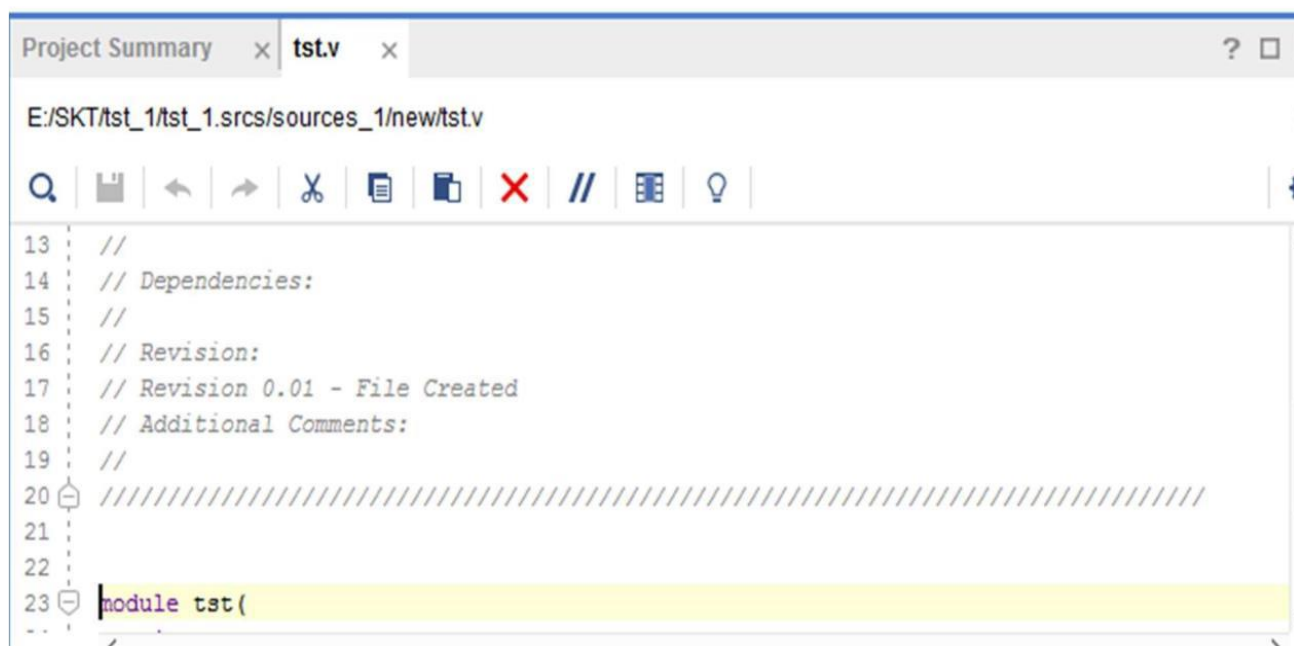


Fig.1.11(b): Writing the source code

14. An example program has been shown here for reference:

After writing the code save the file or press **Ctrl+S**. If there exists a syntax error in your code, it will appear in the source window. Check and remove the syntax errors before moving on to the next step.

After writing the code click on Run simulation icon shown on the left.

After **Run simulation** the top up window will ask **Run Behavioral Simulation**, Click on that.

Example code to get the timing diagrams has been shown below.

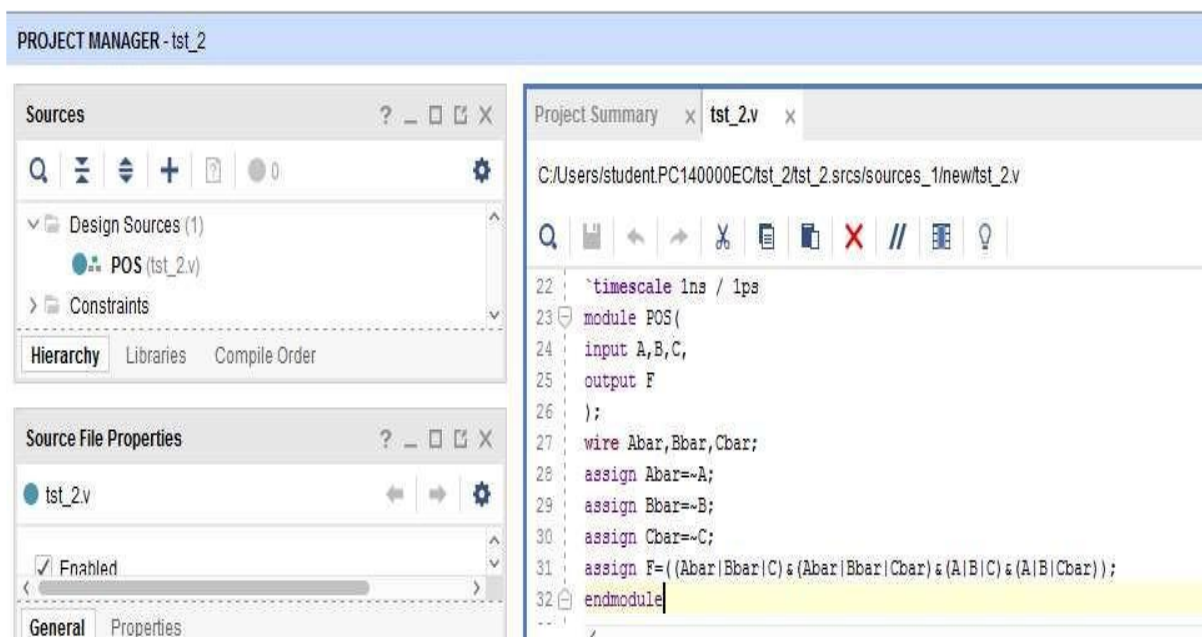


Fig.1.12: Example code

Example-1.0: Write a dataflow Verilog code to realize the given logic function in POS form $y(a, b, c) = \Pi(0, 1, 6, 7)$ and verify the design by simulation.

Solution:

$$y(a, b, c) = (a^- + b^- + c)(a^- + b^- + \bar{c})(a + b + c)(a + b + \bar{c})$$

Verilog Code:

```
module tst_2( input A,B,C, output F);
```

```
wire
```



```

Abar,Bbar,Cbar;
assign Abar=~A;
assign Bbar=~B;
assign Cbar=~C;
assign F=((Abar|Bbar|C)&(Abar|Bbar|Cbar)&(A|B|C)&(A|B|Cbar));
endmodule

```

Run Simulation: Run Behavioral simulation.

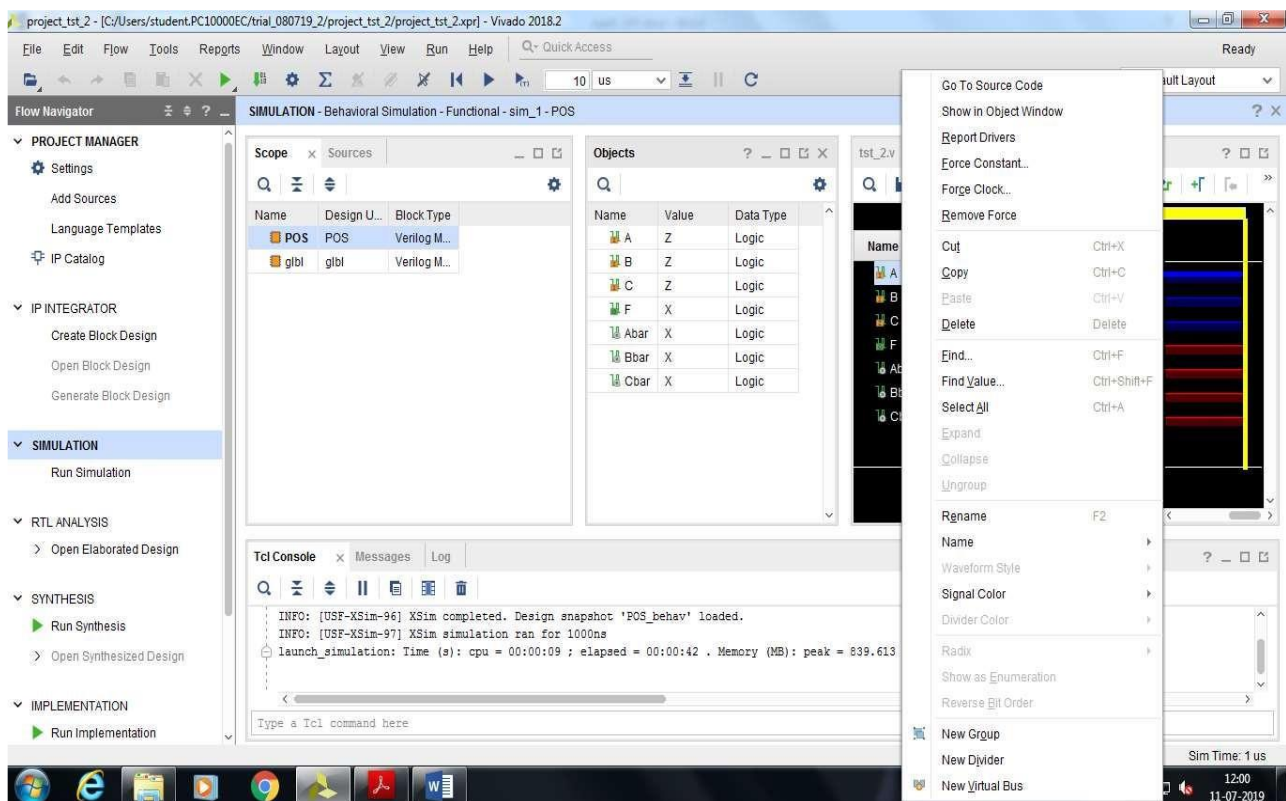


Fig.1.13: Running the simulation

Use force constant, and give the input values, A, B, C in this example. After forcing the values click the icon shown above to run the simulation.

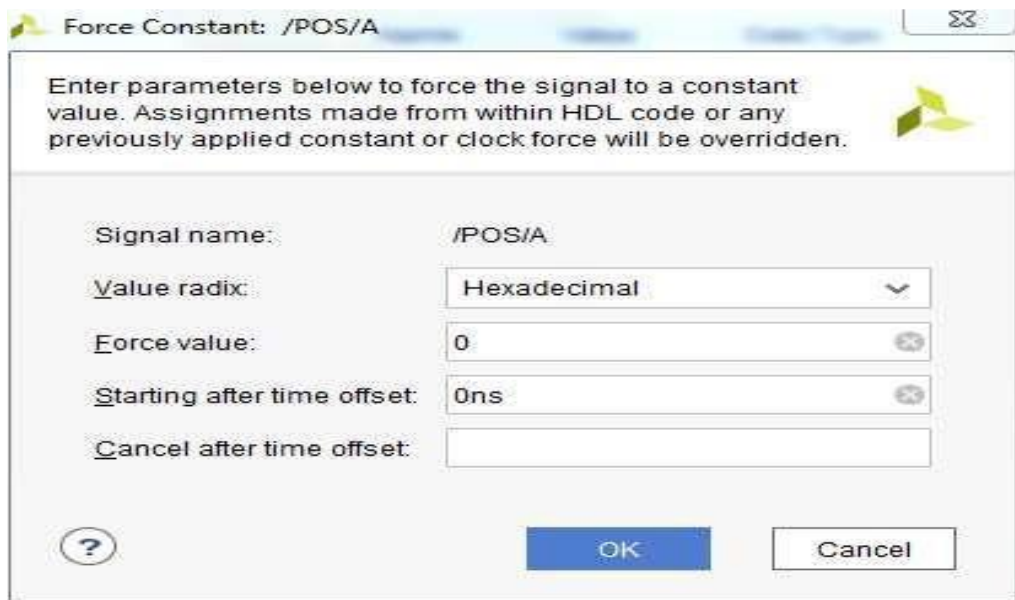


Fig.1.14: Force value as input.

In the figure shown below the input values are A=0; B=1; C=0. F is the output. Corresponding timing diagram is shown.

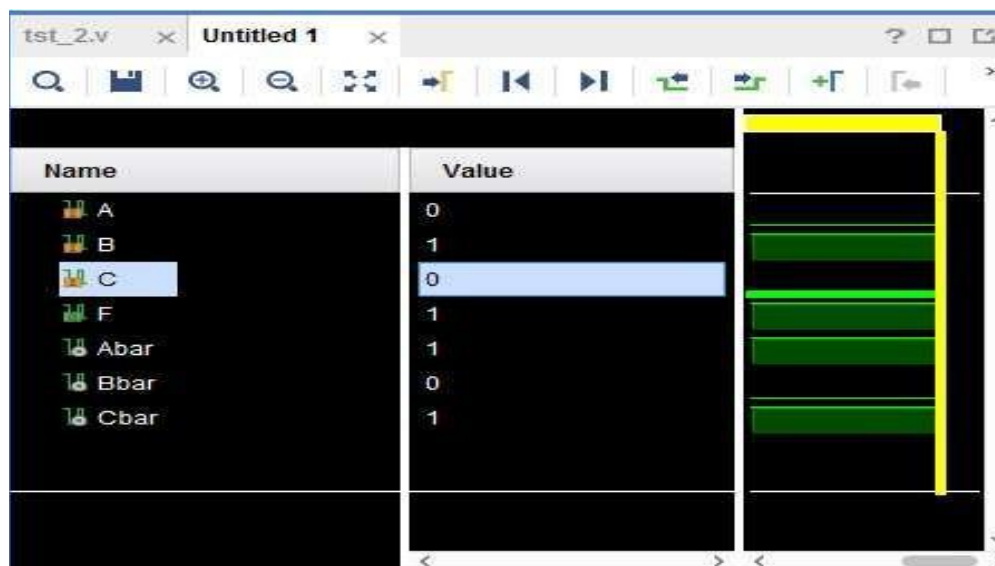


Fig.1.15: Timing diagram

Lab No 2

REALIZE ALL LOGIC GATES

Block Diagram:

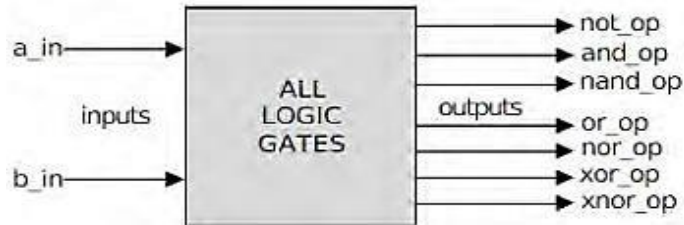


Figure 2.1

Logic Diagram:

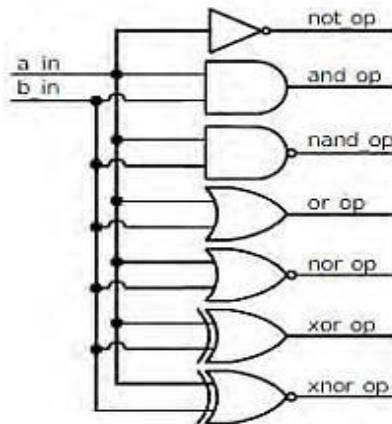


Figure 2.2

Truth Table:

Inputs		Outputs						
a_in	b_in	not_op (a_in)	and_op	nand_op	or_op	nor_op	xor_op	xnor_op
0	0	1	0	1	0	1	0	1
0	1	1	0	1	1	0	1	0
1	0	0	0	1	1	0	1	0
1	1	0	1	0	1	0	0	1

Verilog Code to Realize all logic gates

```
module gates(a_in, b_in, not_op, and_op, nand_op, or_op, nor_op, xor_op, xnor_op);  
input a_in, b_in;  
output not_op, and_op, nand_op, or_op, nor_op, xor_op, xnor_op;  
assign not_op= ~a_in;  
assign and_op=a_in&b_in;  
assign nand_op=~(a_in&b_in);  
assign or_op=a_in|b_in;  
assign nor_op=~(a_in|b_in);  
assign xor_op=a_in^b_in;  
assign xnor_op=~(a_in^b_in);  
endmodule
```

Lab No: 3

SIMPLIFICATION USING K-MAP

Objectives:

In this lab, student will be able to

1. Understand the steps for optimization using K-map.
2. Design minimum cost circuit.
3. Write Verilog code for the simplified expression.

I. Steps for Optimization using K-map:

K-map

- A systematic way of performing optimization.
- Finds a minimum-cost expression for a given logic function by reducing the number of product (or sum) terms needed in the expression, by applying the combining property.
- **Implicant**- A product term that indicates the input valuation(s) for which a given function is equal to 1.
- **Prime Implicant**- An implicant is called a prime implicant if it cannot be combined into another implicant that has fewer literals.
- **Essential prime implicant**-If a prime implicant includes a minterm for which $f = 1$ that is not included in any other prime implicant, then it must be included in the cover and is called as an essential prime implicant.
- The process of finding a minimum-cost circuit involves the following steps:
 1. Generate all prime implicants for the given function f .
 2. Find the set of essential prime implicants.
 3. If the set of essential prime implicants covers all valuations for which $f = 1$, then this set is the desired cover of f . Otherwise, determine the nonessential prime implicants that should be added to form a complete minimum-cost cover.

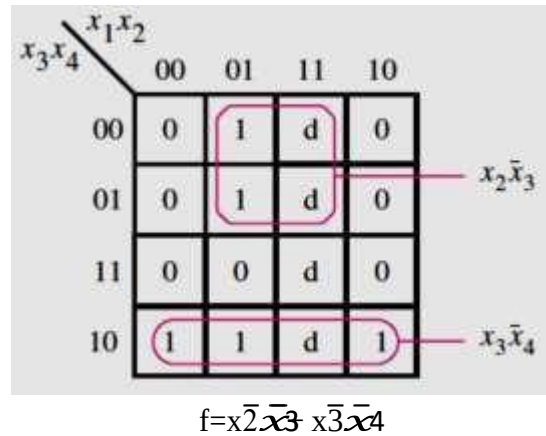
II. Incompletely Specified Functions

- A function that has don't-care condition(s).
- Using the shorthand notation, the function f is specified as
$$f(x_1, \dots, x_4) = \sum m(2, 4, 5, 6, 10) + D(12, 13, 14, 15)$$
where D is the set of don't cares.

Solved Exercise:

Simplify the following function using K-map and write Verilog code to implement this.

$$f(x_1, \dots, x_4) = \sum m(2, 4, 5, 6, 10) + D(12, 13, 14, 15)$$



Verilog code:

```

module example4(x2,x3,x4,f);
    input x2,x3,x4;
    output f;
    assign f=(x2 & ~x3) | (x3 & ~x4);
endmodule

```

Lab Exercises

- Simplify the following functions using K-map and implement the circuit using logic gates.
 - $f(A,B,C,D) = \sum m(2,3,4,5,6,7,10,11,12,15)$
 - $f(A,B,C,D) = \sum m(1,3,4,9,10,12) + D(0,2,5,11)$
- Simplify the following functions using K-map and implement the circuit using logic gates.
 - $f(A,B,C,D) = \prod M(0,1,4,6,8,9,12,14)$
 - $f(A,B,C,D) = \prod M(6,9,10,11,12) + D(2,4,7,13)$
- Simulate a circuit that has four inputs, x_1 , x_2 , x_3 , and x_4 , which produces an output value of 1 whenever three or more of the input variables have the value 1; otherwise, the output has to be 0.

Additional Exercises:

- Simplify the following function using K-map and implement the circuit using logic gates.
 $f(A,B,C,D,E) = \sum m(0,1,8,9,16,17,22,23,24,25)$
- Using only basic gates, simulate a circuit that has four inputs and one output. The output is high if exactly two or exactly three of its variables are equal to 1.

Lab No: 4

MULTILEVEL SYNTHESIS

Objectives:

In this lab, student will be able to

1. Design multilevel NAND and NOR circuits.
 2. Write Verilog code for multilevel circuits.
 3. Use functional decomposition for the synthesis of multilevel circuits.
- **Multilevel NAND and NOR Circuits**
 - Multilevel AND-OR circuits can be realized by a circuit that contains only NAND gates or only NOR gates.
 - Each AND gate is converted to a NAND by inverting its output.
 - Each OR gate is converted to a NAND by inverting its inputs.
 - Each AND gate is converted to a NOR by inverting its inputs.
 - Each OR gate is converted to a NOR by inverting its output.
 - Inversions that are not a part of any gate can be implemented as two-input NAND/NOR gates, where the inputs are tied together.
 - **Functional decomposition-** Complex logic circuit can be reduced by decomposing a two-level circuit into subcircuits, where one or more subcircuits implement functions that may be used in several places to construct the final circuit.

Solved Exercise

Apply functional decomposition for the following function to obtain a simplified circuit and simulate using Verilog.

$$f = \bar{x}_1x_2x_3 + x_1\bar{x}_2x_3 + x_1x_2x_4 + \bar{x}_1\bar{x}_2x_4$$

Factoring x_3 from the first two terms and x_4 from the last two terms, this expression becomes

$$f = (\bar{x}_1x_2 + x_1\bar{x}_2)x_3 + (x_1x_2 + \bar{x}_1\bar{x}_2)x_4$$

Now let $g(x_1, x_2) = \bar{x}_1x_2 + x_1\bar{x}_2$ and observe that

$$\begin{aligned}\bar{g} &= \overline{\bar{x}_1x_2 + x_1\bar{x}_2} \\ &= \overline{\bar{x}_1x_2} \cdot \overline{x_1\bar{x}_2} \\ &= (x_1 + \bar{x}_2)(\bar{x}_1 + x_2) \\ &= x_1\bar{x}_1 + x_1x_2 + \bar{x}_2\bar{x}_1 + \bar{x}_2x_2 \\ &= 0 + x_1x_2 + \bar{x}_1\bar{x}_2 + 0 \\ &= x_1x_2 + \bar{x}_1\bar{x}_2\end{aligned}$$

Then f can be written as

$$f = gx_3 + \bar{g}x_4$$

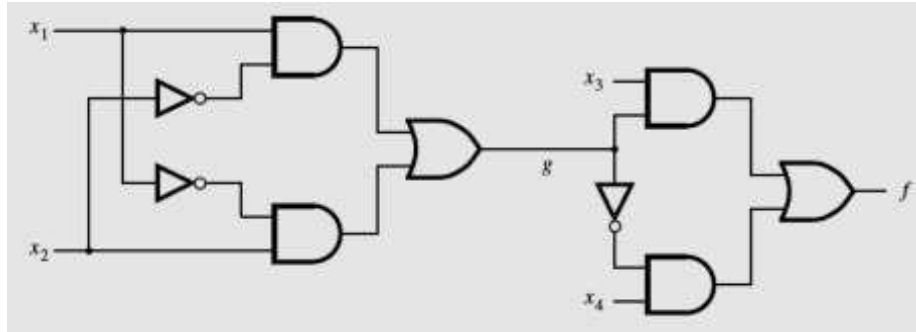


Figure 4.1

Verilog code:

```
module example5(x1,x2,x3,x4,f);
  input x1,x2,x3,x4;
  output f;
  assign g=(x1 & ~x2) | (~x1 & x2);
  assign f=(g & x3) | (~g & x4);
endmodule
```

Lab Exercises

1. Minimize the following expression using K-map and simulate using only NAND gates.
 $f(A,B,C,D) = \pi M(2,6,8,9,10,11,14)$
2. Minimize the following expressions using K-map and simulate using only NOR gates.
 $f(A,B,C,D) = \sum m(0,1,2,5,8,9,10)$
3. Use functional decomposition to find the best implementation of the function and simulate the circuit using Verilog.
 $f(x_1, \dots, x_5) = \sum m(1, 2, 7, 9, 10, 18, 19, 25, 31) + D(0, 15, 20, 26).$
4. Minimize the following expressions using K-map and simulate using NOR gates only.
 $f(A,B,C,D) = \sum m(1,3,5,7,9) + D(6,12,13)$

Additional Exercises

1. Minimize the following expression using K-map and simulate using only NAND gates.
 $f(A,B,C,D) = \prod [(1,3,5,8,9,11,15)+D(2,13)]$
2. Find the minimum cost SOP implementation for the following function f using K-map. $f = A'C'D' + A'C + AB'C' + ACD'$
3. Using functional decomposition find the minimum-cost circuit for the following function f . Assume that the input variables are available in uncomplemented form only. Simulate the circuit using Verilog.
 $f(x_1, \dots, x_4) = \sum m(0, 4, 8, 13, 14, 15).$

Lab No: 5

ARITHMETIC CIRCUITS

Objectives:

In this lab, student will be able to

1. Design arithmetic circuits using combinational logic.
2. Simulate arithmetic circuits using Verilog.

I. Adder circuit:

- **Half adder**- a circuit that implements the addition of only two single bit inputs.
- **Full adder**- a circuit that implements the addition of two single bit inputs and one carry bit.
- **Ripple-carry adder**
 - For each bit position, we can use a full-adder circuit, connected as shown in Fig. 4.1.
 - Carries that are produced by the full-adders propagate to the left.

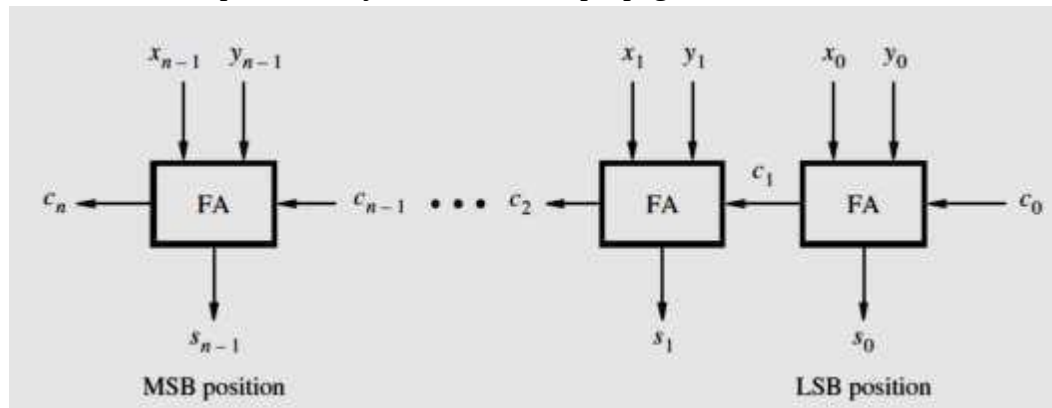


Figure 5.1 An n-bit ripple carry adder

- **Adder/Subtractor unit-**
 - The only difference between performing addition and subtraction is that for subtraction it is necessary to use the 2's complement of one operand.
 - Add/Sub control signal chooses whether addition or subtraction is to be performed.
 - Outputs of the XOR gates represent Y if Add/Sub = 0, and they represent the 1's complement of Y if Add/Sub = 1.
 - Add/Sub is also connected to the carry-in c_0 . This makes $c_0 = 1$ when subtraction is to be performed, thus adding the 1 that is needed to form the 2's complement of Y .
 - When the addition operation is performed, we will have $c_0 = 0$.
 - The circuit is shown in Fig. 5.1

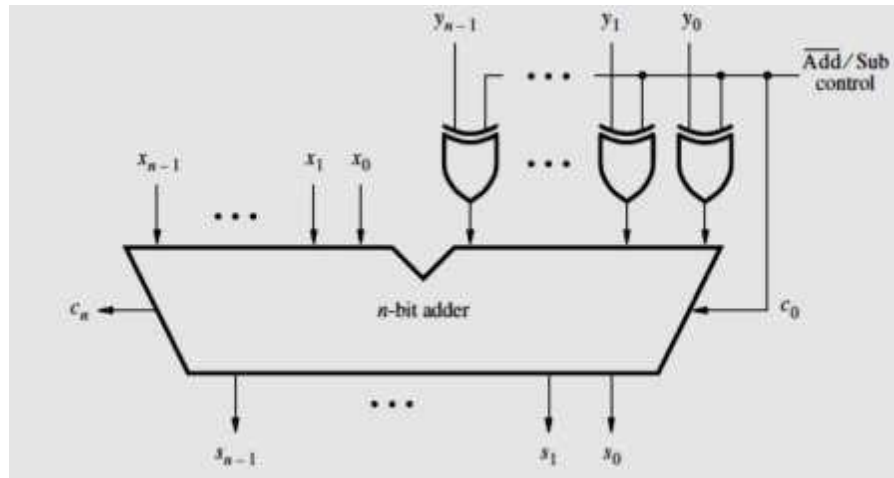


Figure 5.2 Adder/subtractor unit

- **Binary multiplier**

- Multiplication of binary numbers is performed in the same way as in decimal numbers. The multiplicand is multiplied by each bit of the multiplier starting from the least significant bit.
- Each such multiplication forms a partial product. Successive partial products are shifted one position to the left. The final product is obtained from the sum of the partial products.

- **BCD Addition**

In Binary Coded Decimal (BCD) representation each digit of a decimal number is represented by 4-bit binary. When 2 BCD numbers are added,

- If $X + Y \leq 9$, then the addition is the same as the addition of 2 four-bit unsigned binary numbers.
- A correct decimal digit can be generated by adding 6 to the result of a four-bit addition whenever the result exceeds 9 or when the carry is generated.

II. Designing subcircuits in Verilog

- A Verilog module can be included as a subcircuit in another module.
- Both modules must be defined in the same file.
- The general form of a module instantiation statement is given below.

```
module_name [#(parameter overrides)] instance_name (
    .port_name ( [expression] ) {, .port_name ( [expression] ) } );
```

- The **instance_name** can be any legal Verilog identifier and the port connections specify how the module is connected to the rest of the circuit.
- The same module can be instantiated multiple times in a given design provided that each instance name is unique.

- The `#(parameter overrides)` can be used to set the values of parameters defined inside the ***module_name*** module.
- Each ***port_name*** is the name of a port in the subcircuit, and each expression specifies a connection to that port.
- **Named port connections** -The syntax ***.port_name*** is provided so that the order of signals listed in the instantiation statement does not have to be the same as the order of the ports given in the **module** statement of the subcircuit.
- **Ordered port connections**-If the port connections are given in the same order as in the subcircuit, then ***.port_name*** is not needed.

Using Vectored Signals

- Multibit signals are called *vectors*.
- An example of an input vector is
input [3:0] W;
- This statement defines *W* to be a four-bit vector. Its individual bits can be referred to using an index value in square brackets.
- The most-significant bit (MSB) is referred to as *W[3]* and the least-significant bit (LSB) is *W[0]*.

Solved Exercise

Write the Verilog code to implement a 4-bit adder.

Verilog code:

```
module adder4 (carryin, x3, x2, x1, x0, y3, y2, y1, y0, s3, s2, s1, s0, carryout);
    input carryin, x3, x2, x1, x0, y3, y2, y1, y0;
    output s3, s2, s1, s0, carryout;
    fulladd stage0 (carryin, x0, y0, s0, c1);
    fulladd stage1 (c1, x1, y1, s1, c2);
    fulladd stage2 (c2, x2, y2, s2, c3);
    fulladd stage3 (c3, x3, y3, s3, carryout);
endmodule

module fulladd (Cin, x, y, s, Cout);
    input Cin, x, y;
    output s, Cout;
    assign s = x ^ y ^ Cin;
    assign Cout = (x & y) | (x & Cin) | (y & Cin);
endmodule
```

- Separate Verilog module for the ripple carry adder instantiate the fulladd module as a subcircuit.

Lab Exercises

Write behavioral Verilog code to implement the following and simulate

1. Half adder, full adder and decomposed full adder.
2. Four-bit adder using full adders.
3. Four-bit adder/ subtractor using a four-bit adder.
4. 2-bit multiplier using a 2-bit adder and basic gates.
5. Single-digit BCD adder using a four-bit adder(s).

Additional Exercises

1. Design and simulate a circuit that determines how many bits in a six-bit unsigned number are high.
2. Design and write Verilog code for a 2 digit BCD adder.
3. Design a 2-bit multiplier using half adders and logic gates. Write the Verilog code to implement this design.

Lab No: 6

MULTIPLEXERS

Objectives:

In this lab, student will be able to

1. Understand the concept of multiplexers.
2. Learn more about the behavioral style of Verilog programming.
3. Design and implement simple multiplexers.
4. Design and implement large multiplexers using small multiplexers.

I. Multiplexers

- The multiplexer has a number of data inputs, one or more select inputs, and one output.
- It passes the signal value on one of the data inputs to the output.
- A multiplexer that has N data inputs, $w_0, \dots, w_{N-1}$, requires $\log_2 N$ select inputs.
- Fig. 6.1a shows the graphical symbol for a 2-to-1 multiplexer.
- The functionality of a multiplexer can be described in the form of a truth table. Fig. 6.1b shows the functionality of a 2-to-1 multiplexer.

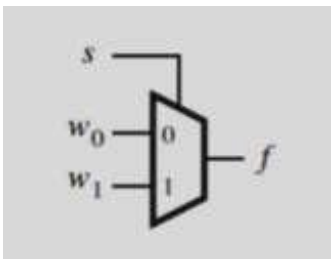


Figure 6.1a Graphical symbol

s	f
0	w_0
1	w_1

Figure 6.1b Truth table

The Conditional Operator

- In a logic circuit, it is often necessary to choose between several possible signals or values based on the state of some condition.
- Verilog provides a conditional operator ($?:$) which assigns one of the two values depending on a conditional expression.

Syntax of conditional operator

```
conditional_expression ? true_expression : false_expression
```

- If the conditional expression evaluates to 1 (true), then the value of true_expression is chosen; otherwise, the value of false_expression is chosen.

The case Statement

- The general form of a **case** statement is given below.

```

case (expression)
    alternative1: begin
        statement;
    end
    alternative2: begin
        statement;
    end
    [default: begin
        statement;
    end]
endcase

```

- The bits in expression, called as controlling expression, are checked for a match with each alternative.
- The first successful match causes the associated statements to be evaluated.
- Each digit in each alternative is compared for an exact match of the four values 0, 1, x, and z.
- A special case is the **default** clause, which takes effect if no other alternative matches.
- The **casex** statement reads all z and x values as don't cares.

Functions and Tasks

- The purpose of a **function** is to allow the code to be written in a modular fashion without defining separate modules.
- A **function** is defined within a module, and it is called either in a continuous assignment statement or in a procedural assignment statement inside that module.
- A **function** can have more than one input, but it does not have an output, because the **function** name itself serves as the output variable.
- Function general form

```

function [range | integer] function_name;
    [input declarations]
    [parameter, reg, integer declarations]
begin
    statement;
end
endfunction

```

Verilog task

- A task is declared by the keyword **task** and it comprises a block of statements that ends with the keyword **endtask**.
- The task must be included in the module that calls it.
- It may have input and output ports.
- The task ports are used only to pass values between the module and the task.

Solved Exercise

Write behavioral Verilog code for 2 to 1 multiplexer using **always** and **conditional** operators.

Verilog code:

```
module mux2to1 (w0, w1, s, f);  
    input w0, w1, s;  
    output f;  
    reg f;  
    always @(w0 or w1 or s)  
        f = s ? w1 : w0;  
endmodule
```

Lab Exercises

1. Write behavioral Verilog code for a 2 to 1 multiplexer using the **if-else** statement. Use this to write the hierarchical code for a 4 to 1 multiplexer.
2. Write behavioral Verilog code for a 4 to 1 multiplexer using **conditional** operator. Use this to write the hierarchical code for a 16 to 1 multiplexer.
3. Write behavioral Verilog code for an 8 to 1 multiplexer using **case** statement. Use this along with a 2 to 1 multiplexer to write the hierarchical code for a 16 to 1 multiplexer.
4. Write behavioral Verilog code for a 2 to 1 multiplexer using **function**. Use this to write the hierarchical code for a 4 to 1 multiplexer.

Additional Exercises

1. Write behavioral Verilog code for an 8 to 1 multiplexer using an **if-else** statement. Use this to write the hierarchical code for a 32 to 1 multiplexer.
2. Write behavioral Verilog code for a 4 to 1 multiplexer using **function**. Use this to write the hierarchical code for a 16 to 1 multiplexer.

Lab No : 7

DECODERS AND ENCODERS

Objectives:

In this lab, student will be able to

1. Learn the concept of decoders, encoders and priority encoders.
2. Write Verilog code for decoders, decoder trees and encoders.

Decoders

- Decoder circuits are used to decode the encoded information.
- A binary decoder, depicted in Fig. 8.1, is a logic circuit with n inputs and 2^n outputs.
- Each output corresponds to one valuation of the inputs, and only one output is asserted at a time.
- The decoder also has an enable input En , that is used to disable the outputs; if $En = 0$, then none of the decoder outputs is asserted.
- If $En = 1$, the valuation of $w_{n-1} \cdots w_1 w_0$ determines which of the outputs is asserted.
- Larger decoders can be built using smaller decoders referred to as decoder tree.

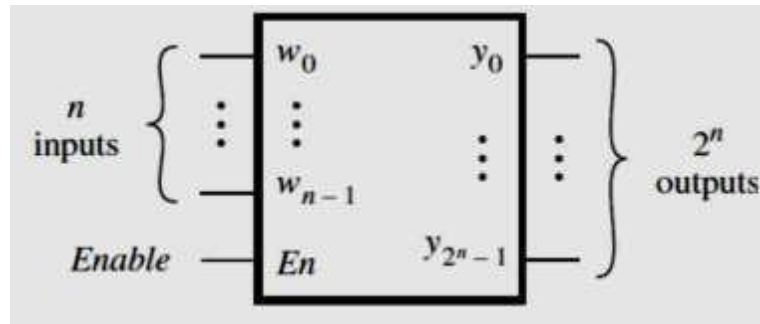


Figure 7.1 Decoder

Binary Encoders

- A binary encoder encodes information from 2^n inputs into an n -bit code, as indicated in Fig. 8.2.
- Exactly one of the input signals should have a value of 1, and the outputs present the binary number that identifies which input is equal to 1.

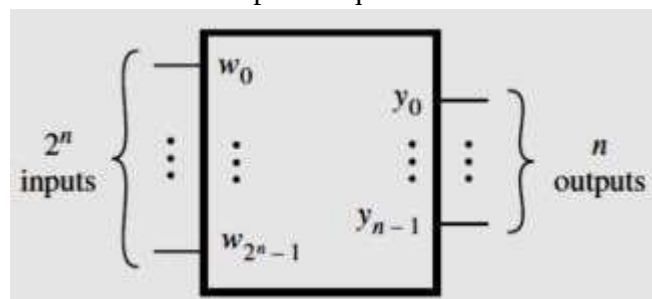


Figure 7.2 Encoder

Priority Encoder

- In a priority encoder, each input has a priority level associated with it.
- When an input with a high priority is asserted, the other inputs with lower priority are ignored. Since it is possible that none of the inputs is equal to 1, an output, z, is provided to indicate this condition.
- The truth table for a 4-to-2 priority encoder is shown in Fig. 8.3.

w ₃	w ₂	w ₁	w ₀	y ₁	y ₀	z
0	0	0	0	d	d	0
0	0	0	1	0	0	1
0	0	1	x	0	1	1
0	1	x	x	1	0	1
1	x	x	x	1	1	1

Figure 7.3 Truth table for a 4 to 2 priority encoder

Case Statement:

- Verilog provides variants of the **case** statement that treat the z and x values in a different way.
- The **casez** statement treats all z values as don't cares.
- The **casex** statement treats all z and x values as don't cares.

Solved exercise

Write behavioral Verilog code for 2 to 4 binary decoder using **for** loop.

Verilog code:

```
module dec2to4 (W, Y, En);
    input [1:0] W;
    input En;
    output [0:3] Y;
    reg [0:3] Y;
    integer k;
    always @(W or En)
        for (k = 0; k <= 3; k = k+1)
            if ((W == k) && (En == 1))
                Y[k] = 1;
            else
                Y[k] = 0;
endmodule
```

Lab Exercises

1. Write behavioral Verilog code for a 2 to 4 decoder with active-high enable input and active high output using the **if-else** statement. Using the 2 to 4 decoders above, design a 3 to 8 decoder and write the Verilog code for the same.
2. Write behavioral Verilog code for a 3 to 8 decoder with active-high enable input and active high output using **for** loop. Using the 3 to 8 decoders above, design a 4 to 16 decoder and write the Verilog code for the same.
3. Write behavioral Verilog code for a 2 to 4 decoder with active-high enable input and active low output using **case** statement. Using the 2 to 4 decoders above, design a 4 to 16 decoder with active-high enable input and active low output and write the Verilog code for the same.
4. Write behavioral Verilog code for a 4 to 2 priority encoder using **case** statement.
5. Write behavioral Verilog code for 16 to 4 priority encoder using **for** loop.

Additional Exercises

1. Write behavioral Verilog code for a 2 to 4 decoder with active-high enable input and active high output using **case** statement. Using the 2 to 4 decoders above, design a 4 to 16 decoder with active-low enable input and active high output and write the Verilog code for the same.
2. Write behavioral Verilog code for a 3 to 8 decoder with active-high enable input and active low output using the **if-else** statement. Using 3 to 8 decoders and a 2 to 4 decoder, design a 5 to 32 decoder with active-high enable input and active low output and write the Verilog code for the same.

Lab No: 8

COMPARATORS AND CODE CONVERTERS

Objectives:

In this lab, student will be able to

1. Learn the concept of comparators and code converters.
2. Write Verilog code to simulate comparators and code converters.

Arithmetic Comparison Circuits

- Compare the relative magnitude of two binary numbers.

Code Converters

- Convert from one type of input representation to a different output representation.

Parameters

- A parameter associates an identifier name with a constant.
- Using the following declaration, the identifier n can be used in place of the number 4.

parameter n = 4;

Verilog Procedural Statements

- Rather than using gates or logic expressions, circuits can be specified in terms of their behavior.
- Also called sequential statements.
- Procedural statements are evaluated in the order in which they appear in the code whereas concurrent statements are executed in parallel.
- Verilog syntax requires that procedural statements should be inside an **always** block.

Always Block

- An **always** block is a construct that contains one or more procedural statements.
- It has the form

```
always @(sensitivity_list)
[begin]
    [procedural assignment statements]
    [if-else statements]
    [case statements]
    [while, repeat, and for loops]
    [task and function calls]
[end]
```

- The **sensitivity_list** is a list of signals that directly affect the output results generated by the **always** block.
- If the value of a signal in the sensitivity list changes, then the statements inside the **always** block are evaluated in the order presented.

Variables:

- A variable can be assigned a value and this value is retained until it is overwritten in a subsequent assignment statement.
- There are two types of variables, **reg**, and an **integer**.
 - The keyword **reg** does not denote a storage element or register. In Verilog code, **reg** variables can be used to model either combinational or sequential parts of a circuit.
 - **Integer** variables are useful for describing the behavior of a module, but they do not directly correspond to nodes in a circuit.

The if-else Statement

- The general form of the **if-else** statement is given below.

```

if (expression1)
begin
    statement;
end
else if (expression2)
begin
    statement;
end
else
begin
    statement;
end

```

- If expression1 is true, then the first statement is evaluated.
- When multiple statements are involved, they have to be included inside a **begin-end** block.
- The **else if** and **else** clauses are optional.
- Verilog syntax specifies that when **else if** or **else** are included, they are paired with the most recent unfinished **if** or **else if**.

for Loop

- the general form of **for** loop

```
for (initial_index; terminal_index; increment)
begin
    statement;
end
```

- The **initial_index** is evaluated once, before the first loop iteration and typically performs the initialization of the **integer** loop control variable.
- In each loop iteration, the begin-end block is performed, and then the increment statement is evaluated.
- Finally, the **terminal_index** condition is checked, and if it is True (1), then another loop iteration is done.

Solved Exercise

Write Verilog code to simulate a 1-bit equality comparator.

Verilog code:

```
module compare(A, B, AeqB);
    input A, B;
    output reg AeqB;
    always @(A,B)
    begin
        AeqB=0;
        if (A==B)
            AeqB=1;
    end
endmodule
```

Lab exercises

1. Design a 5-bit comparator using only logic gates. Write behavioral Verilog code to simulate the design.
2. Using **for** loop, write behavioral Verilog code to convert an N bit grey code into equivalent binary code.
3. Write behavioral Verilog code to simulate a code converter that converts a decimal digit from 8, 4,-2,-1 code to BCD.
4. Write and simulate the Verilog code for a 4-bit comparator using 2-bit comparators.

Additional exercises

1. Write behavioral Verilog code to design an n-bit adder with carry out and overflow signals.
2. Write Verilog code for a 4-bit signed comparator circuit with three outputs *AequaltoB*, *AlessthanB*, *AgreaterthanB* designed using full adders and other necessary gates.
3. Using **for** loop, write behavioral Verilog code to convert an N bit binary number into an equivalent grey code.
4. Write Verilog code to perform BCD to excess-3 code conversion. Use K-map to derive the simplified expressions for excess-3 code.

Lab No: 9
FLIP FLOPS AND REGISTERS

Objectives:

In this lab. student will be able to

1. Learn different types of Flip Flops.
2. Understand the concept of triggering of flip flops and different types of reset.
3. Understand the concept of registers and shift registers.
4. Write Verilog code for Flip Flops and registers.

I. Flip Flops and Registers

Flip Flops:

- A flip flop circuit can maintain a binary state until directed by an input signal to switch the state.
- Major differences among various types of flip flops are in the number of inputs they process and in the manner in which the inputs affect the binary state.

Triggering of Flip-Flops:

- The state of a flip flop is switched by a momentary change in the input signal which is called triggering the flip flop.

Positive Edge and Negative Edge:

- A positive clock source remains at 0 during the interval between pulses and goes to 1 during the occurrence of a pulse.
- The pulse transition from 0 to 1 is called a **positive edge** and return from 1 to 0 is called a **negative edge**.

Registers:

- A register is a group of binary cells suitable for holding binary information.

Shift Registers:

- A register capable of shifting its binary information either to the right or to the left is called a shift register.

II. Verilog Constructs for Storage Elements

- The Verilog keywords **posedge** and **negedge** are used to implement edge-triggered circuits.
- The keyword **posedge** specifies that a change may occur only on the positive edge of Clock.

- The keyword **negedge** specifies that a change may occur only on the negative edge of Clock.

Blocking and Non-Blocking Assignments

Blocking

- A Verilog compiler evaluates the statements in an **always** block in the order in which they are written.
- If a variable is given a value by a blocking assignment statement, then this new value is used in evaluating all subsequent statements in the block.
- Denoted by the '=' symbol
- Example


```
Q1 = D;
Q2 = Q1;
```

- The above statement results in Q2=Q1=D

Non-Blocking

- Verilog also provides a non-blocking assignment, denoted with '<='.
- All non-blocking assignment statements in an **always** block are evaluated using the values that the variables have when the **always** block is entered.
- Thus, a given variable has the same value for all statements in the block.
- The meaning of non-blocking is that the result of each assignment is not seen until the end of the **always** block.
- Example


```
Q1 <= D;
Q2 <= Q1;
```
- The variables Q1 and Q2 have some value at the start of evaluating the **always** block, and then they change to a new value concurrently at the end of the **always** block.

Flip-Flops with Clear Capability

- By using a particular sensitivity list and a specific style of an **if-else** statement, it is possible to include clear (or preset) signals on flip-flops.

Solved Exercise:

Write behavioral Verilog code for positive edge-triggered D FF with synchronous reset.

Verilog Code:

```
module flipflop (D, Clock, Resetn, Q);  
    input D, Clock, Resetn;  
    output Q;  
    reg Q;  
    always @(posedge Clock)  
    if (!Resetn)  
        Q <= 0;  
    else  
        Q <= D;  
endmodule
```

Lab Exercises

1. Write behavioral Verilog code for a positive edge-triggered D FF with asynchronous active high reset.
2. Write behavioral Verilog code for a negative edge triggered T FF with asynchronous active low reset.
3. Write behavioral Verilog code for a positive edge-triggered JK FF with synchronous active high reset.
4. Write structural Verilog code for a 5-bit register.
5. Write structural Verilog code for a 6-bit shift register.

Additional Exercises

1. Write Verilog code for an N bit register.
2. Write Verilog code for an N bit shift register.
3. Design and simulate a sequential circuit with two JK flip-flops A and B, two inputs x and y and one output z. The flipflop input functions and the circuit output functions are
$$\begin{array}{lll} JA = Bx' + B'y & KA = B'x'y' & \\ JB = A'x & KB = A' + xy & z = A'xy + Bx'y' \end{array}$$

4. Design and simulate a shift register with the parallel load that operates according to the following function table:

Shift	Load	Register Operation
0	0	Shift left
0	1	Load parallel data
1	X	No change

Lab No: 10

COUNTERS

Objectives:

In this lab, student will be able to

1. Learn the concept of synchronous/asynchronous up/down counters.
2. Learn the concept of ring and Johnson counters.
3. Write Verilog code for different types of counters.

Counters:

- A counter is essentially a register that goes through a predetermined sequence of states upon the application of input pulses.
- A binary counter with a reverse count is called a binary down counter.

Ripple Counters

- Also called as asynchronous counters.
- The CP inputs of all flip flops (except the first) are triggered not by the incoming pulses, but by the transition that occurs in other flip flops.
- Four-bit binary ripple counter is shown in Fig. 11.1

Synchronous Counters

- The input pulses are applied to the CP input of all the flip flops.
- The common pulse triggers all flip flops simultaneously.
- The change of state of a particular flip flop is dependent on the present state of other flip flops.
- For synchronous sequential circuits the design procedure is as follows:
 3. From the given information (word description/state diagram/timing diagram/other pertinent information) about the circuit, obtain the state table.
 4. Determine the number of flip flops needed.
 5. From the state table, derive the circuit excitation and output tables.
 6. Derive the circuit output functions and the flip flop input functions by simplification.
 7. Draw the logic diagram.

Ring Counter

- Circular shift register with only one flip flop being set at any particular time, all others are cleared.
- N bit ring counter will have N states and requires N flip flops.
- Fig. 11.2 shows a 4-bit ring counter using decoder and counter.

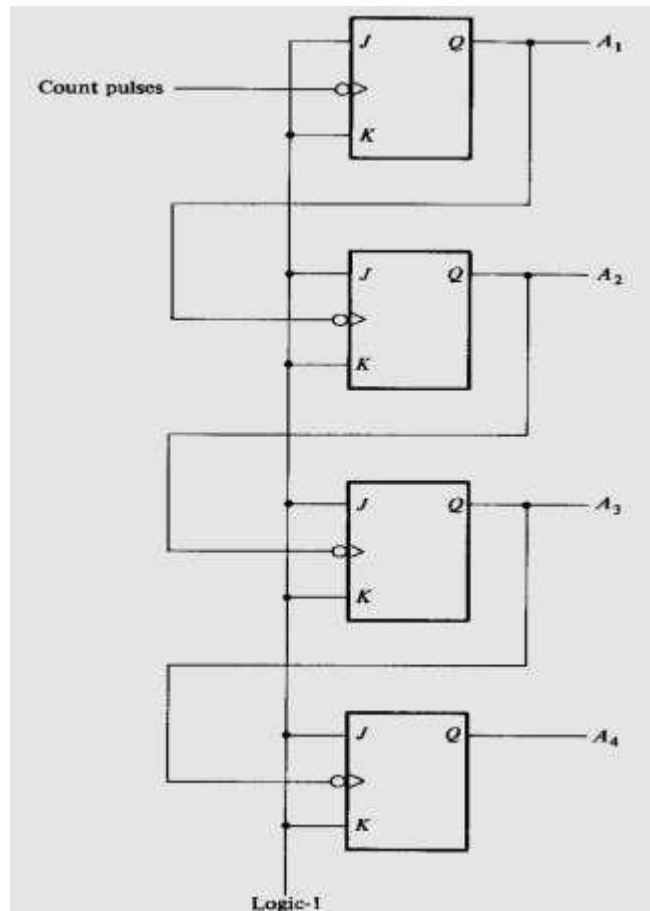


Figure 10.1 4-bit binary ripple counter

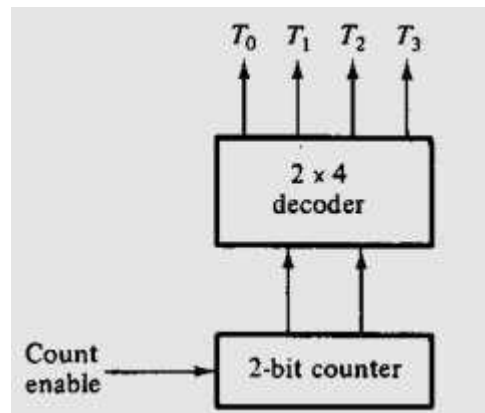


Figure 10.2 4-bit ring counter

Johnson counter or Switch tail ring counter

- Circular shift register with the complement output of the last flip flop connected to the input of the first flip flop.
- k-bit switch tail ring counter with $2k$ decoding gates provide outputs for $2k$ timing signals.

- k-bit Johnson counter requires k flip flops.
- Fig. 11.3 shows a 4-bit Johnson counter

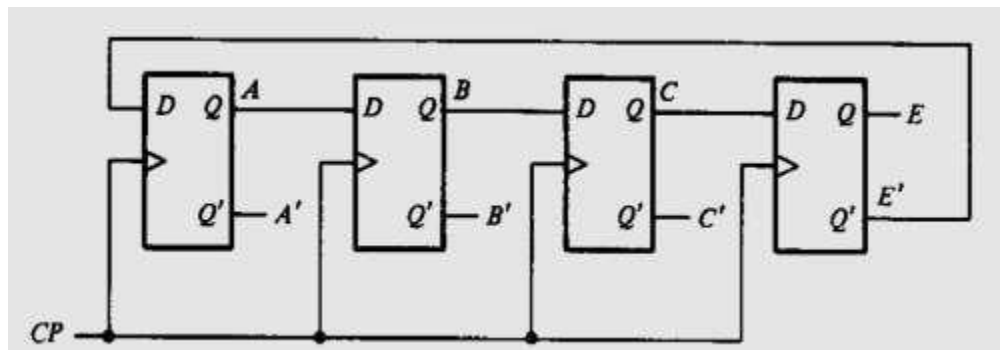


Figure 10.3 4-bit Johnson counter

Solved Exercise

Write Verilog code for a 2-bit asynchronous up counter.

```
module tff1(T, Clock, Q);
    input T, Clock;
    output Q;
    reg Q;
    always @(negedge Clock)
    if (!T)
        Q <= Q;
    else
        Q <= ~Q;
endmodule

module twobit (clock, Q);
    input clock;
    output [1:0]Q;
    tff1 f1(1,clock,Q[0]);
    tff1 f2(1,Q[0],Q[1]);
endmodule
```

Lab Exercises

- Design and simulate the following counters
 - 4-bit ring counter.
 - 5 bit Johnson counter.
 - 4 bit asynchronous up counter

- d) 4 bit synchronous up counter
- e) 4 bit synchronous up/down counter with a control input $up/\overline{down}$. If $up/\overline{down} = 1$, then the circuit should behave as an up counter. If $up/\overline{down} = 0$, then the circuit should behave as a down counter.

Additional Exercises

- Design and simulate a sequential circuit which produces a high output for every 6th clock pulse.
- Design and simulate a synchronous Mod 11 counter. Treat the unused states as don't-care conditions.
- Assume a 4-bit signal A and a clock as inputs and a 4-bit signal Y as output. Design a circuit which repeats the operations cyclically with a clock as follows:

Clock(input)	Y(output)
1 st clock cycle	rotate A by 1 bit to the left
2 nd clock cycle	rotate A by 2 bit to the left
3 rd clock cycle	rotate A by 3 bit to the left
4 th clock cycle	complement of A

And then repeats in the same pattern.

- Design and simulate a synchronous counter using T FFs with a control input w which operates in the manner given. When w=0, the counter should follow the repeated binary sequence: 0, 2, 3, 4, 6 and when w=1, the counter should follow the repeated binary sequence: 6, 4, 3, 2, 0.
- Assume two single bit signals A, B and a Clock as inputs and a 2 bit signal Y as output. Design and simulate a sequential circuit which repeats the operations cyclically with clock as follows:

Clock(input)	Y(output)
1st clock cycle	A'
2 nd clock cycle	A' + 1
3 rd clock cycle	B'
4 th clock cycle	B' + 1
5 th clock cycle	2A
6 th clock cycle	2B
7 th clock cycle	A' + B'
8 th clock cycle	A + B

And then repeats in the same pattern.

Lab No: 11

REALIZE 4 BIT BINARY TO GRAY CONVERTER

THEORY: Gray code is a non-weighted code. In these codes while traversing from one step to another step only one bit in the code group changes. In case of Gray Code two adjacent code numbers differ from each other by only one bit.

The problem with natural binary codes is that, with physical, mechanical switches, it is very unlikely that switches will change states exactly in synchrony. For example, in the transition between 011 to 100, all three switches change state. In the brief period while all are changing, the switches will read some spurious position. Even without key bounce, the transition might look like 011 — 001 — 101 — 100. When the switches appear to be in position 001, the observer cannot tell if that is the "real" position 001, or a transitional state between two other positions. If the output feeds into a sequential system, possibly via combinational logic, then the sequential system may store a false value. The reflected binary code or Gray code solves this problem by changing only one switch at a time.

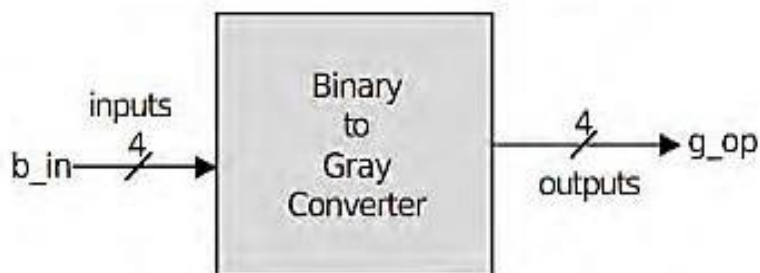
Conversion of Binary to Gray code:

1. The MSB of the gray code will be equal to the MSB of the given binary code.
2. The second bit of the gray code will be exclusive-or of the first and second bit of the given binary number, i.e. if both the bits are same the result will be 0 and if they are different the result will be 1.
3. The third bit of gray code will be equal to the exclusive-or of the second and third bit of the given binary number. This process continues until we get the last bit.

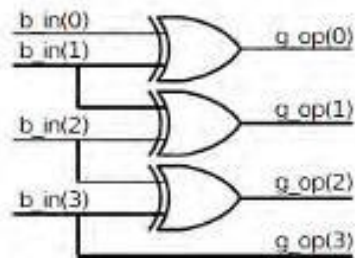
Conversion of Gray to Binary code:

1. The MSB of the binary code will be equal to the MSB of the given gray code.
2. Take the next bit down in the Gray code and the most significant bit of the binary and do a logical XOR and this becomes the next bit in the binary code.
3. Take the next bit down in the Gray code and the next bit down in the binary code and do a logical XOR and this becomes the next bit in the binary code.
4. Continue this process until we get the last bit.

Block Diagram:



Logic Diagram and expressions:



Boolean Expressions

$$g_op(3) = b_in(3)$$

$$g_op(2) = b_in(3) \oplus b_in(2)$$

$$g_op(1) = b_in(2) \oplus b_in(1)$$

$$g_op(0) = b_in(1) \oplus b_in(0)$$

Truth Table:

Decimal	Inputs				Outputs			
	Binary				Gray			
	b_in(3)	b_in(2)	b_in(1)	b_in(0)	g_op(3)	g_op(2)	g_op(1)	g_op(0)
0	0	0	0	0	0	0	0	0
1	0	0	0	1	0	0	0	1
2	0	0	1	0	0	0	1	1
3	0	0	1	1	0	0	1	0
4	0	1	0	0	0	1	1	0
5	0	1	0	1	0	1	1	1
6	0	1	1	0	0	1	0	1
7	0	1	1	1	0	1	0	0
8	1	0	0	0	1	1	0	0
9	1	0	0	1	1	1	0	1
10	1	0	1	0	1	1	1	1
11	1	0	1	1	1	1	1	0
12	1	1	0	0	1	0	1	0
13	1	1	0	1	1	0	1	1
14	1	1	1	0	1	0	0	1
15	1	1	1	1	1	0	0	0

VERILOG CODE of 4 bit binary to gray converter:

```
module binary_gray(b_in, g_op);
input [3:0] b_in;
output [3:0] g_op;

assign g_op[3] = b_in[3];
assign g_op[2] = b_in[3] ^ b_in[2];
assign g_op[1] = b_in[2] ^ b_in[1];
assign g_op[0] = b_in[1] ^ b_in[0];
endmodule
```


Lab No: 12

SIMPLE PROCESSOR DESIGN

Objectives:

In this lab student will be able to

1. Learn the concept of bus structure.
2. Understand the concept of a simple processor and a bit counting circuit.
3. Write Verilog code to implement bus structure, simple processor and bit counting circuit.

Bus Structure:

- When a digital system contains a number of n-bit registers, to transfer data from any register to any other register, a simple way of providing the desired interconnectivity is to connect each register to a common set of n wires, which are used to transfer data into and out of the registers. This common set of wires is usually called a bus.
- If common paths are used to transfer data from multiple sources to multiple destinations, it is necessary to ensure that only one register acts as a source at any given time and other registers do not interfere.

There are two arrangements for implementing the bus structure.

- Using Tri-State Drivers
- Using Multiplexers

Using Tri-State Drivers to Implement a Bus:

- Consider a system that contains k n-bit registers, R1 to Rk. Figure 12.1 shows how these registers can be connected using tri-state drivers to implement the bus structure. The data outputs of each register are connected to tri-state drivers. When selected by their enable signals, the drivers place the contents of the corresponding register onto the bus wires. The enable signals are generated by a control circuit.
- In addition to registers, in a real system, other types of circuit blocks would be connected to the bus. The figure shows how n bits of data from an external source can be placed on the bus, using the control input Extern.
- It is essential to ensure that only one circuit block attempts to place data onto the bus wires at any given time. The control circuit must ensure that only one of the tri-state driver enables signals, R1out, . . . , Rkout, is asserted at a given time. The control circuit also produces the signals R1in, . . . , Rkin, which determines when data is loaded into each register.
- In general, the control circuit could perform a number of functions, such as transferring the data stored in one register into another register and controlling the processing of data in

various functional units of the system. Figure 12.1 shows an input signal named Function that instructs the control circuit to perform a particular task. The control circuit is synchronized by a clock input, which is the same clock signal that controls the k registers.

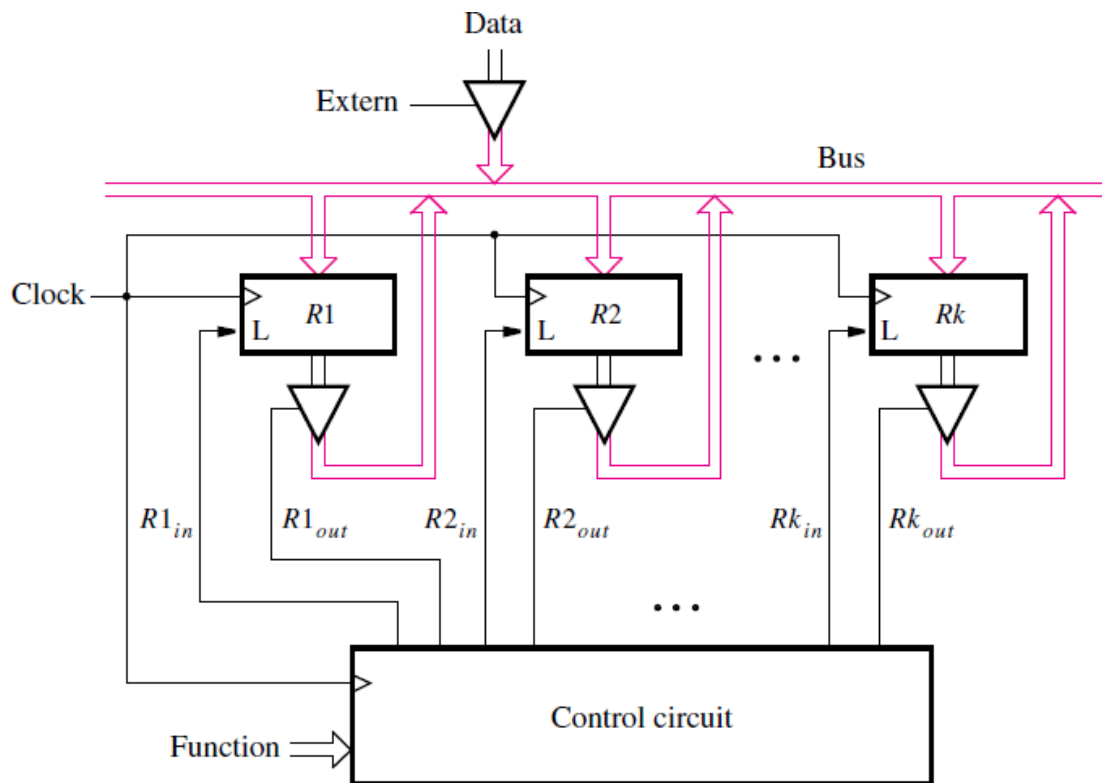


Fig 12.1 A digital system with k registers.

Solved Exercise

Consider a system that has three registers, $R1$, $R2$, and $R3$. The control circuit performs a single function—it swaps the contents of registers $R1$ and $R2$, using $R3$ for temporary storage. The required swapping is done in three steps, each needing one clock cycle. In the first step, the contents of $R2$ are transferred into $R3$. Then the contents of $R1$ are transferred into $R2$. Finally, the contents of $R3$, which are the original contents of $R2$, are transferred into $R1$. To transfer the contents of one register into another bus is used. The control circuit for this task can be explained in the form of a finite state machine as shown in Figure 12.2. Its state table is shown in Table 12.1

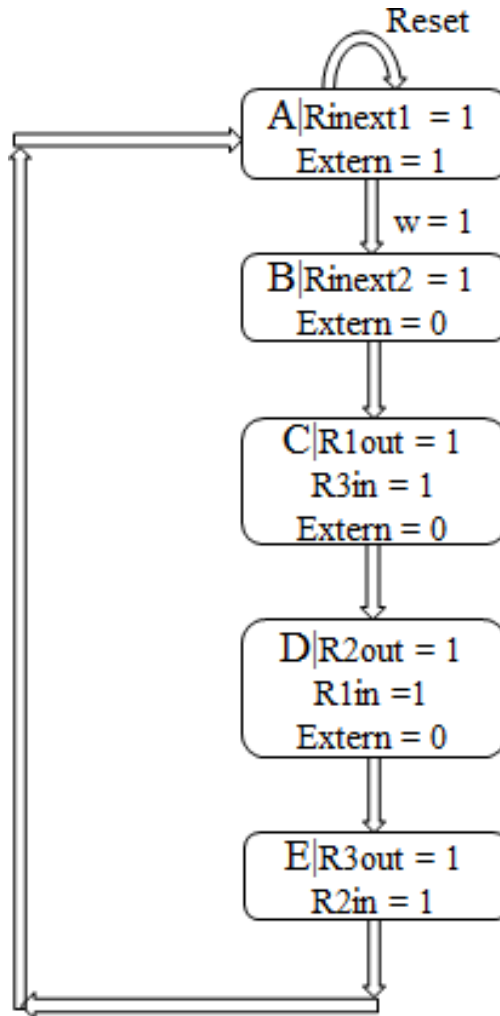


Fig. 12.2

Initially, the bus is loaded with the data when Extern=1. The initial state is A corresponding to load the data from the bus to register R1. To initiate the state transition from state A to state B, an input signal w is made equal to 1. In state A and state B, Extern = 1 to load two different data to the bus and then from the bus to the registers R1 and R2. The states B, C and D change their states if Extern = 0. From B, it goes to C state where R1 is copied to R3. From C it goes to state D where R2 is copied to R1. From D, the next state is E to copy R3 to R2 and an output Done = 1 to indicate that swap is completed. From E the next state is the initial state A. All transfers are taking place through the bus.

Table 12.1

PS	NS		Outputs								
	Extern=1	Extern=0	R1next1	R1next2	R1in	R1out	R2in	R2out	R3in	R3out	done
A	B	A	1	0	0	0	0	0	0	0	0
B	B	C	0	1	0	0	0	0	0	0	0
C	C	D	0	0	0	1	0	0	1	0	0
D	D	E	0	0	1	0	0	1	0	0	0
E	A	A	0	0	0	0	1	0	0	1	1

Verilog code

```
module regn (R, L, Clock, Q);
parameter n = 8;
input [n-1:0] R;
input L, Clock;
output [n-1:0] Q;
reg [n-1:0] Q;
always @(posedge Clock)
if (L)
Q <= R;
endmodule // Code for 8-bit register

module swap1 (Resetn, Clock, w, Data, Extern, R1, R2, R3, BusWires, Done);
parameter n = 8;
input Resetn, Clock, w, Extern;
input [n-1:0] Data;
output [n-1:0] BusWires ,R1, R2, R3 ;
reg [n-1:0] BusWires, R1, R2, R3;
output Done;
wire R1in, R1out, R2in, R2out, R3in, R3out, RinExt1, RinExt2;
reg [2:0] y, Y;
parameter [2:0] A = 3'b000, B = 3'b001, C = 3'b010, D = 3'b011, E = 3'b100;
// Define the next state combinational circuit for FSM
always @(w or y)
begin
case (y)
A: if (w) Y = B;
```

```

else Y = A;

B: Y = C;

C: Y = D;

D: Y = E;

E: Y = A;

endcase

end

// Define the sequential block for FSM

always @(negedge Resetn or posedge Clock)

begin

if (Resetn == 0) y <= A;

else y <= Y;

end

// Define outputs of FSM

assign RinExt1 = (y == A);

assign RinExt2 = (y == B);

assign R3in = (y == C);

assign R1out = (y == C);

assign R2out = (y == D);

assign R1in = (y == D);

assign R3out = (y == E);

assign R2in = (y == E);

assign Done = (y == E);


always @(Extern or R1out or R2out or R3out)

if (Extern) BusWires = Data;

else if (R1out) BusWires = R1;

```

else if (R2out) BusWires = R2;

else if (R3out) BusWires = R3;

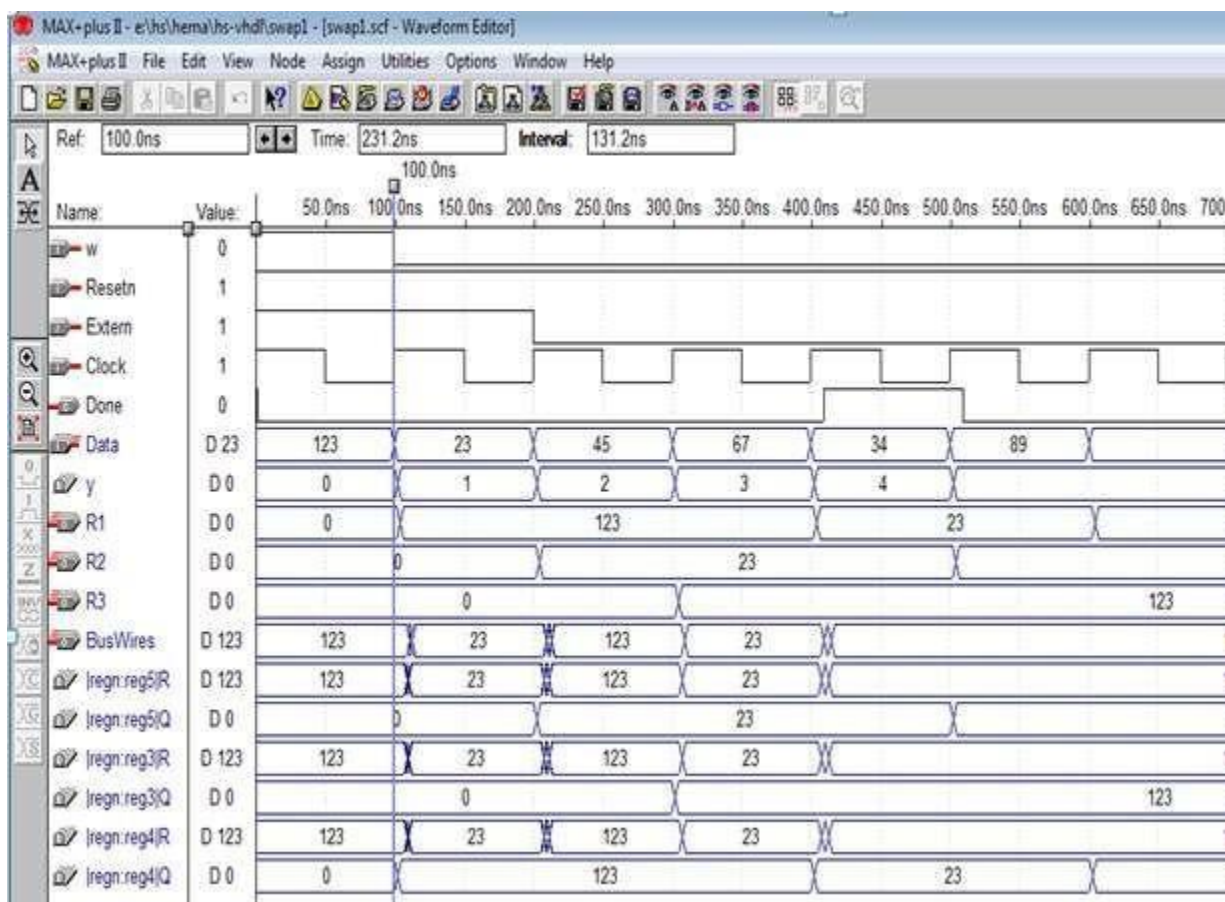
regn reg3 (BusWires, R3in, Clock, R3);

regn reg4 (BusWires, RinExt1 | R1in, Clock, R1);

regn reg5 (BusWires, RinExt2 | R2in, Clock, R2);

endmodule

Output:



Lab Exercises

1. Implement the swap example shown in the solved exercise using multiplexers.
2. Simulate a simple processor that can perform the following functions:

Operation	Function performed
Load $Rx, Data$	$Rx \leftarrow Data$
Move Rx, Ry	$Rx \leftarrow [Ry]$
Add Rx, Ry	$Rx \leftarrow [Rx] + [Ry]$
Sub Rx, Ry	$Rx \leftarrow [Rx] - [Ry]$

Additional Exercises

1. Simulate a bit counting circuit.

References:

1. Stephen Brown and Zvonko Vranesic, “Fundamentals of digital logic with Verilog design”, Tata MGH publishing Co.Ltd., 3rd edition, 2014.
2. M.Morris Mano, “Digital design”, PHI Pvt. Ltd., 2nd edition, 2000

Viva Questions

1. What is combinational logic?

Ans: **Combinational** logic is a type of digital logic which is implemented by Boolean **circuits**, where the output is a pure function of the present input only.

2. What are the applications of decoders?

Ans: Decoders are greatly used in applications where the particular output or group of outputs to be activated only on the occurrence of a specific combination of input levels.

Ex: a) Binary to decimal decoder

b) Address decoder

c) Instruction decoder

3. What is encoder and decoder?

Ans: An **encoder** is a device, circuit, transducer, software program, algorithm or person that converts information from one format or code to another, for the purposes of standardization, speed, secrecy, security or compressions.

if there are 2^n input lines, and at most only one of them will ever be high, the binary code of this 'hot' line is produced on the n -bit output lines.

A **binary decoder** is a combinational logic circuit that converts binary information from the n coded inputs to a maximum of 2^n unique outputs. They are used in a wide variety of applications, including data demultiplexing, seven segment displays, and memory address decoding.

4. What is the difference between encoder and priority encoder?

Ans: The **Priority Encoder** solves the problems faced by encoder by allocating a priority level to each input. The *priority encoders'* output corresponds to the currently active input which has the highest priority. So when an input with a higher priority is present, all other inputs with a lower priority will be ignored.

5. What is meant by sensitivity list?

Ans: The **sensitivity list** must include all input signals used by an always block to properly model combinational logic.

6. What is the advantage of gray code?

Ans: **Advantage** of grey **code** over binary is only one-bit changes for each step. This will be useful in circuits that are sensitive to glitches. Example: When you use grey **code** to trim output impedance of IO blocks, there won't be a huge jump in **codes** momentarily that would cause reflection.

7. What is de-multiplexer?

Ans: **Demultiplex (DEMUX)** is the reverse of the multiplex (MUX) process – combining multiple unrelated analog or digital signal streams into one signal over a single shared medium, such as a single conductor of copper wire or fiber optic cable.

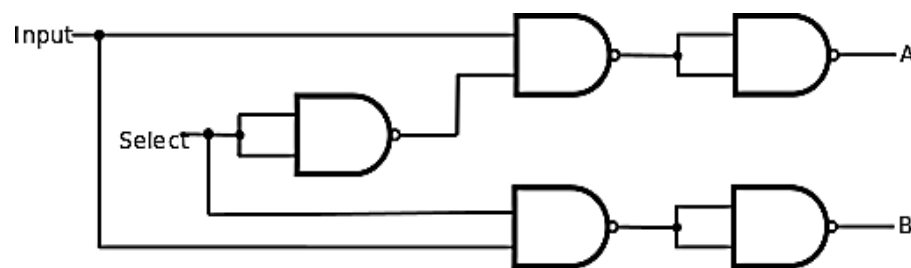
8. Mention about the application of de-multiplexer.

Ans: a) **communication system**: mux and demux both are used in communication system to carry out the process of data transmission. A De-multiplexer receives the output signals from the multiplexer and at the receiver end it converts them back to the original form.

b) **Arithmetic Logic Unit**: The output of the ALU is fed as an input to the De-multiplexer, and the output of the demultiplexer is connected to a multiple register. The output of the ALU can be stored in multiple registers.

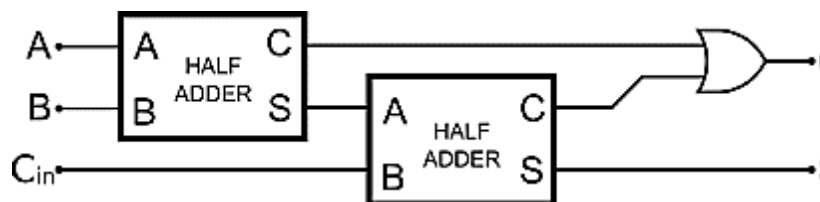
c) **Serial to Parallel Converter**: This converter is used to reconstruct parallel data. In this technique, serial data is given as an input to the De-multiplexer at a regular interval, and a counter is attached to the demultiplexer at the control input to detect the data signal at the output of the demultiplexer. When all data signals are stored, the output of the demux can be read out in parallel.

9. How do you realize a de-multiplexer using NAND gates?



10. Implement a full adder using two half adders.

Ans:



11. Give some applications of adders.

Ans: Full **adder** reduces circuit complexity.

It can be used to construct a ripple carry counter to add an n-bit number. Thus it is used in the ALU also. It is used in Processor chip like Snapdragon, Exynos or Intel pentium for CPU part .

12. What is flip-flop?

Ans: Flip-flop is a 1-bit storing element.

13. How many types of flip-flops are used?

Ans: 4 types of flip –flop, S-R, J-K, D, T

14. What is disadvantage of SR flip-flop?

Ans: When both the input is one then it gives invalid output.

15. What is disadvantage of JK flip-flop?

Ans: race around condition.

16. What is race around condition?

Ans: when pulse width is more than signal width then for signal change of pulse width many no of times signal changes its state that is called race around condition.

17. To remove race around condition what we use?

Ans: master slave Flip-flop.

18. What is the characteristic equation for T flip-flop?

Ans: $Q = TQ' + QT'$

19. D flip-flop is used for?

Ans: D flip flop is used for creating Delay.

20. what is full form of T flip-flop?

Ans: Toggle Flip flop

21. Design a mod-n counter with reset input.

22. Design a synchronous mod-3 counter.

23. Design and implement a mod-7 asynchronous BCD counter with reset input.

24. Design and implement 3-bit binary counter on FPGA.

25. Write a program in VHDL and verilog to implement a mod-10 BCD counter with suitable delay on FPGA.

26. Display 000-010-010-101-110-101-000 on FPGA.

27. Design and implement on FPGA, a 3-bit BCD counter with asynchronous reset input.

28. Write a HDL program for asynchronous binary counter using CASE statement only.

29. Generate the RTL schematic for an up/down BCD counter.

30. Synthesize and simulate the HDL program for 5-bit binary counter.

31. Compare the area required for a synchronous and asynchronous BCD counter RTL schematic.

32. Design and implement a RING counter.